TEKsystems Global Services | WORKFORCE DEVELOPMENT

QUALITY ASSURANCE INTRODUCTION



DAY 3: QUALITY IN PRACTICE

- Triage and debugging: logs, stack traces, and isolating defects
- Actionable bug reports, test reports, and quality metrics
- Tests in a CI/CD pipeline with GitHub Actions
- QA antipatterns: flaky tests and the habits that cause them

Yesterday you built tests that find problems. Today is about what happens next.

© 2026 by Innovation In Software Corporation

WHEN TESTS FIND PROBLEMS

Finding a problem is only the start. Someone has to confirm it's real, work out where it lives, decide how urgent it is, and get it to the right person with enough evidence to fix it fast.

That process is **triage**, and QA is usually at the center of it. Good triage is the difference between a fix in an hour and a ticket that bounces between teams for a week.



© 2026 by Innovation In Software Corporation

SYMPTOMS VS. CAUSES

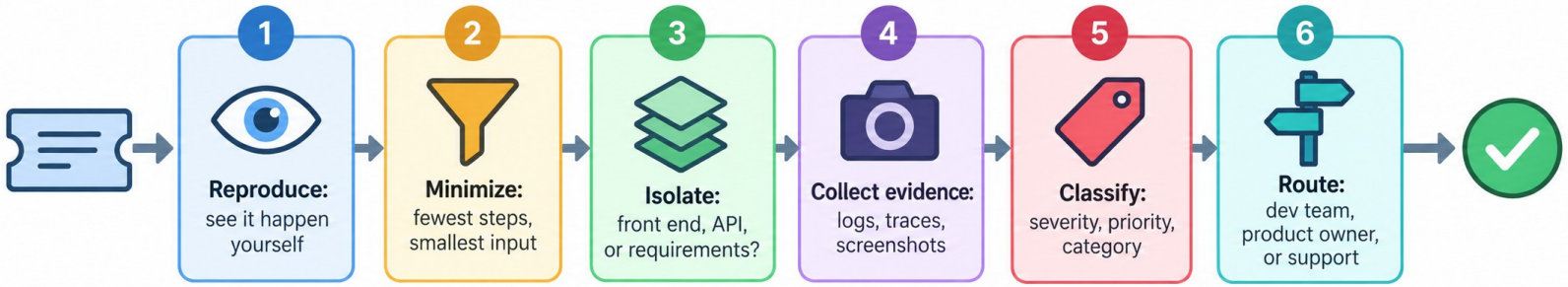
Users describe **symptoms**: "search does nothing," "my total is wrong," "I got locked out." They rarely know the cause, and their description of what they did is often incomplete.

Your job is to turn a symptom into a **reproducible**, **isolated** defect with evidence. That means finding out exactly what happened, where, and why, before anyone writes a line of fix.



© 2026 by Innovation In Software Corporation

THE TRIAGE PROCESS



© 2026 by Innovation In Software Corporation

DEBUGGING TOOLS

Tool	Where	Shows you
DevTools Console	Browser, `F12` → Console	JavaScript errors and warnings in the page
DevTools Network	Browser, `F12` → Network	Every request: URL, status, request body, response body
Server log	The server's terminal or log file	Each request with its status and timing, plus errors with stack traces
Direct API call	Postman or `curl`	Whether the API is right **without** the front end in the way

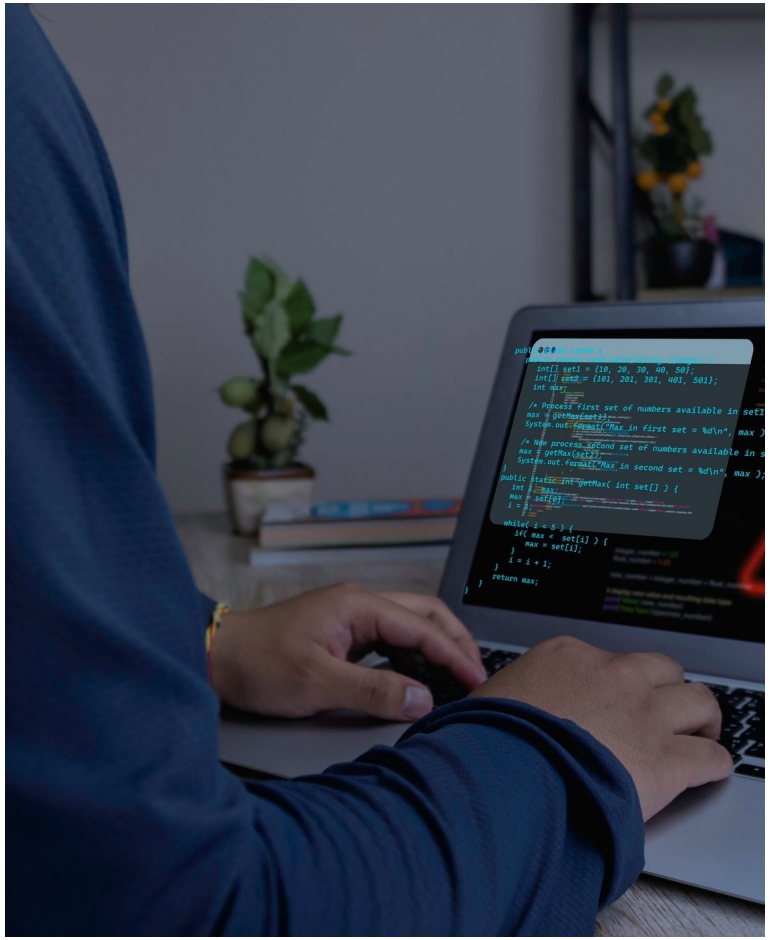
Each tool sees one layer. Together they tell the whole story.

© 2026 by Innovation In Software Corporation

DEVTOOLS: THE CONSOLE

The browser console shows errors from the page's own JavaScript. When a button "does nothing," the console often says why: a script error, a failed request, or an unhandled rejection.

- Open it with `F12` (or right-click → Inspect) and choose **Console**
- Red messages are errors; yellow ones are warnings
- Reproduce the problem with the console open so you see errors as they happen



© 2026 by Innovation In Software Corporation

DEVTOOLS: THE NETWORK TAB

The Network tab records every request the page makes. For each one you can see the URL, method, status code, how long it took, and exactly what was sent and received.

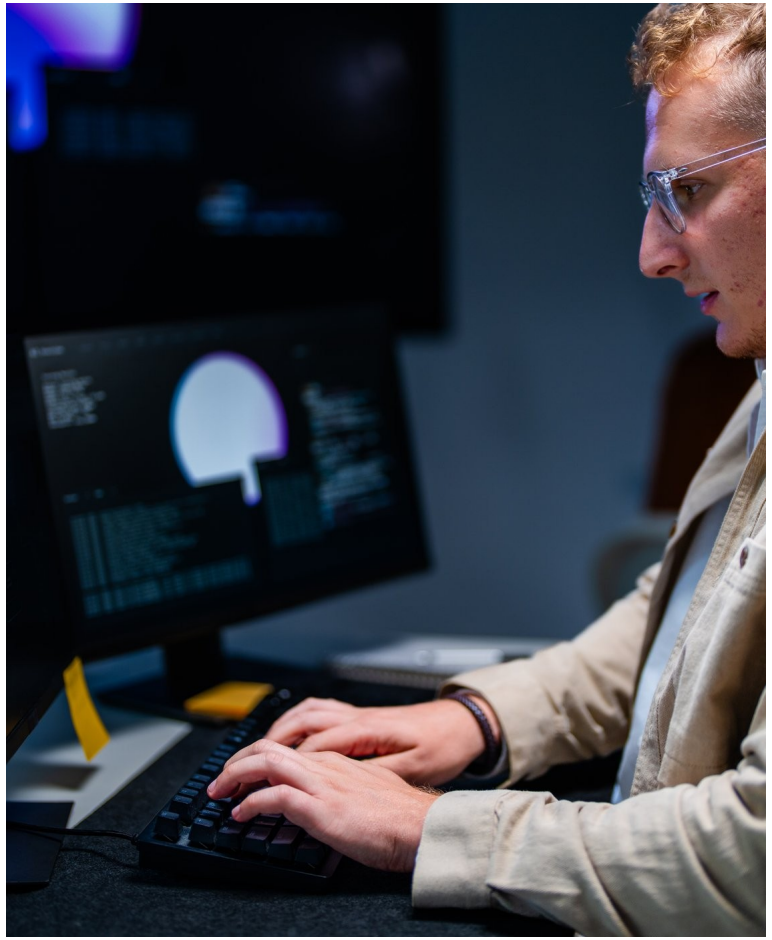
- A 4xx or 5xx status points to a problem in the request or on the server
- Compare the **response body** with what the page shows
- If the response is right but the page is wrong, the bug is in the front end

© 2026 by Innovation In Software Corporation

WHAT IS A LOG?

A log is the system's diary: a time-stamped record of what it did and what went wrong. When users can't tell you what happened, the log often can.

Good logs answer who, what, when, and how it ended. They're the first place to look when a problem can't be reproduced, and the only place to look for problems that happened yesterday.



© 2026 by Innovation In Software Corporation

ANATOMY OF A LOG LINE

Part	Example	Tells you
Timestamp	`2026-10-05T08:15:49.130Z`	Exactly when; compare with the user's story
Level	`WARN`	How serious the event was
Message	`login failed`	What happened
Fields	`email=...`, `failedAttempts=5`	The details you'll search and filter on

© 2026 by Innovation In Software Corporation

LOG LEVELS

Level	Means	Example
ERROR	Something failed and needs attention	A request crashed with an exception
WARN	Something unusual but handled	A failed login, a rejected discount code
INFO	Normal, notable events	A request completed, a user logged in
DEBUG	Detailed internals for developers	Variable values, decision points

Search for ERROR first, then read the lines just before it to see what led there.

© 2026 by Innovation In Software Corporation

WHAT IS A STACK TRACE?

When code crashes, it leaves a **stack trace**: the error type, a message, and the chain of function calls that led to the failure, most recent first.

It looks intimidating, but you only need three things from it: what went wrong, where in **your** code it happened, and who called that code. Paste it into bug reports as text, so developers can search and copy it.

© 2026 by Innovation In Software Corporation

READING A STACK TRACE

```
TypeError: Cannot read properties of undefined (reading 'price')   ← 1. what went wrong
    at summarize (C:\app\server.js:93:22)                            ← 2. first frame in OUR code
    at C:\app\server.js:210:14                                       ← 3. who called it
    at Layer.handle (C:\app\node_modules\express\lib\...)            ← 4. framework: usually ignore
```

- Read top to bottom: the error type and message first
- Find the first `file:line:column` in **your** code
- Frames inside `node_modules` are the framework calling your code; the bug is rarely there

© 2026 by Innovation In Software Corporation

READING A STACK TRACE

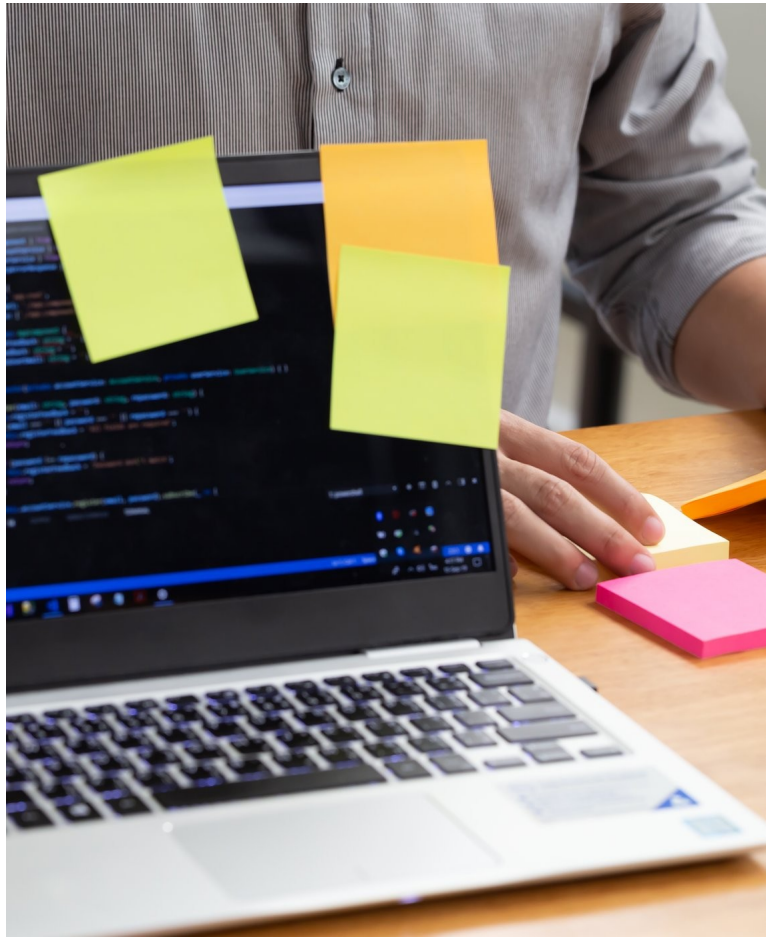


© 2026 by Innovation In Software Corporation

REPRODUCING A DEFECT

You can't fix what you can't see. Reproducing means making the problem happen yourself, on purpose, as reliably as possible.

- Ask exactly what the user did: steps, input, account, browser, device, time
- Start from a known state, such as fresh data and a new browser session
- Try the user's exact steps first, then vary them
- Record how often it happens: 5 out of 5, or 1 out of 10



© 2026 by Innovation In Software Corporation

WHEN IT WON'T REPRODUCE

"Cannot reproduce" is a result, not a dead end. Something is different between your attempt and the user's. Find it.

Look at	For example
Data	A specific book, a specific code, an item in the cart
Environment	Browser, device, screen size, network speed
Account and state	Logged in or out, previous actions, the time since the last action
Timing	Double-clicks, slow responses, actions in a different order
Logs	What the server recorded at the user's reported time

© 2026 by Innovation In Software Corporation

MINIMIZE

Once it reproduces, strip it down. Remove steps and simplify the input until you have the **smallest** case that still shows the problem.

- Can you skip logging in? Use one item instead of three?
- Which part of the input matters? Try one character at a time
- A pattern in the minimized cases often points straight at the cause

Minimal steps make bug reports shorter, fixes faster, and regression tests easier to write.

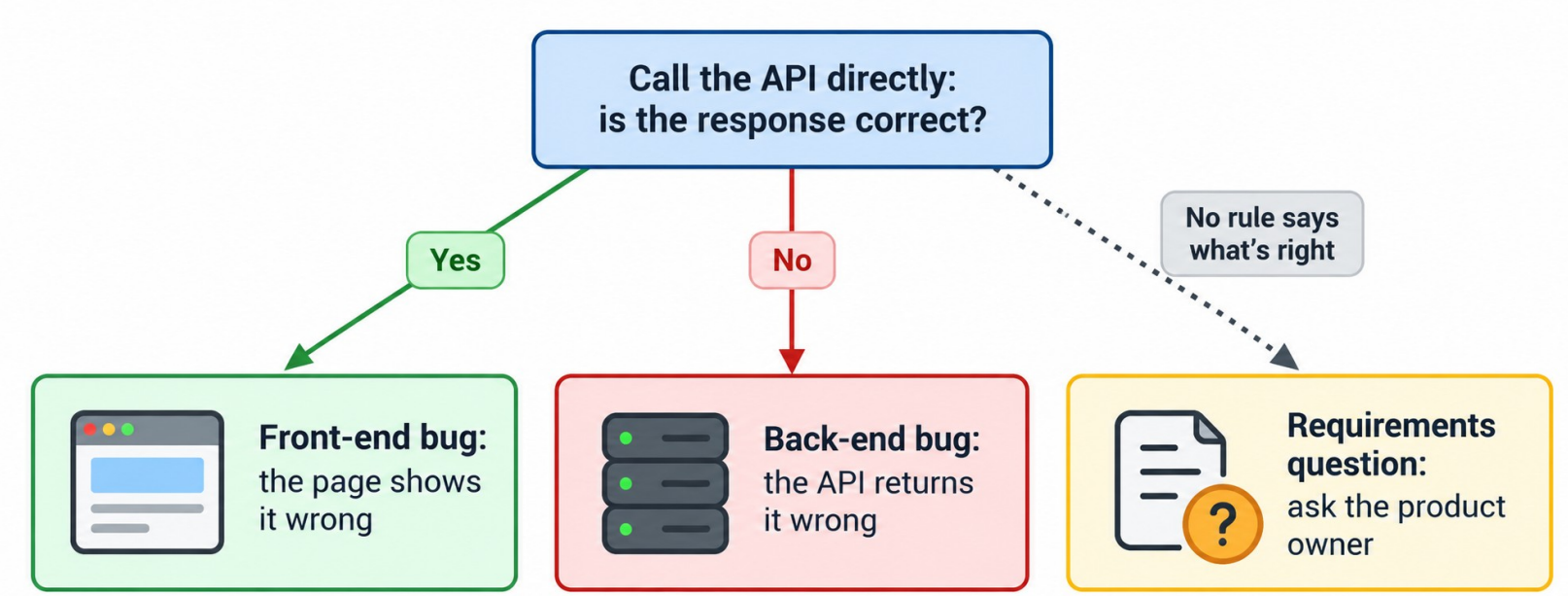
© 2026 by Innovation In Software Corporation

ISOLATE THE LAYER

A web app has layers, and each bug lives in one of them. Isolating the layer sends the bug to the right person with the right evidence.

© 2026 by Innovation In Software Corporation

ISOLATE THE LAYER



© 2026 by Innovation In Software Corporation

CALL THE API DIRECTLY

The fastest isolation trick: take the browser out of the picture and send the same request yourself.

```
curl "http://localhost:3000/api/books?q=dune"
```

- **Same wrong result:** the bug is in the back end
- **Correct result, but the page is wrong:** the bug is in the front end
- **Correct by the code, but surprising:** check the requirements

© 2026 by Innovation In Software Corporation

SEVERITY VS. PRIORITY

	Severity	Priority
Measures	**Impact** on the system or user	**Urgency** to fix
Set by	QA, from evidence	Product owner and team, from business needs
Scale	Critical, High, Medium, Low	P1 (now) to P4 (someday)
Question	"How bad is it?"	"How soon must it be fixed?"

© 2026 by Innovation In Software Corporation

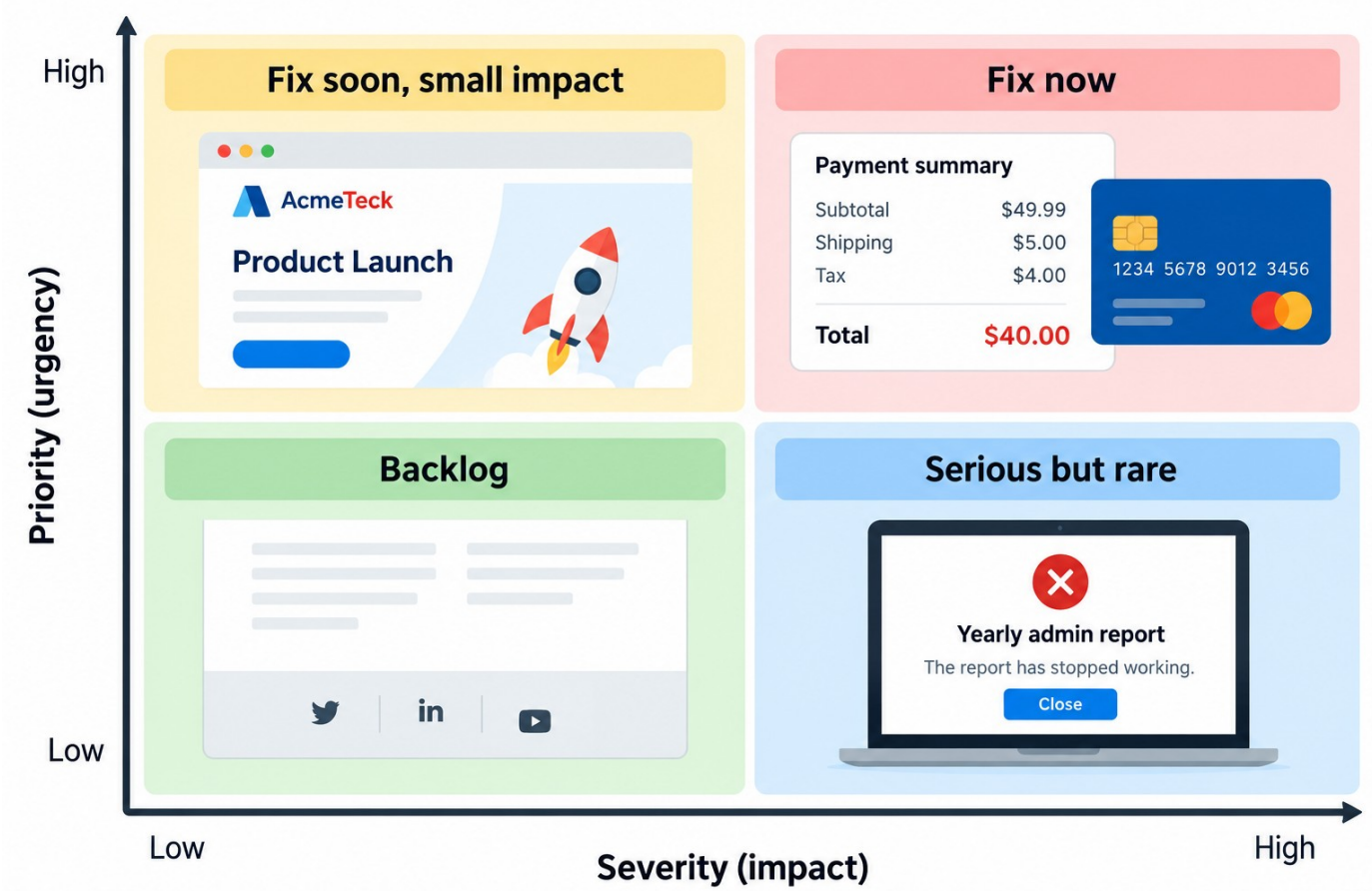
WHEN SEVERITY AND PRIORITY DISAGREE

The two usually move together, but not always.

- **High severity, low priority:** a crash in an admin report that runs once a year
- **Low severity, high priority:** the company logo misspelled on the home page during launch week
- **Both high:** customers are charged the wrong amount at checkout

© 2026 by Innovation In Software Corporation

WHEN SEVERITY AND PRIORITY DISAGREE



© 2026 by Innovation In Software Corporation

CATEGORIZING DEFECTS

Categories show patterns across many bugs and help route each one to the right people.

Category	Example
Functional	A rule from the spec isn't enforced
UI	The page shows stale or wrong information
Data	Wrong values stored or returned
Security	Private data exposed; protections bypassed
Usability	Works, but confusing or unfriendly
Accessibility	Can't be used with a keyboard or screen reader
Requirements	The spec is missing, vague, or contradictory

© 2026 by Innovation In Software Corporation

TRIAGE OUTCOMES

Not every ticket ends with "new bug." Each outcome has a different next step and a different owner.

Outcome	Means	Next action
New defect	Real, not reported before	Write a bug report; send it to the dev team
Duplicate	Same root cause as an existing bug	Link it; add the new evidence
Works as designed	Matches the spec	Explain it to support
Requirements question	The spec is silent or questionable	Send it to the product owner
Cannot reproduce	You can't make it happen	Ask the reporter for specific missing details

© 2026 by Innovation In Software Corporation

DUPLICATES ARE EVIDENCE

A duplicate isn't a waste of time. A second customer hitting a known bug proves it's affecting real people, and their report may reveal a new way to trigger it.

- Link the new ticket to the existing bug
- Add any new steps, data, or impact
- Revisit priority: a "rare" bug with five reports isn't rare

© 2026 by Innovation In Software Corporation

NOT EVERY TICKET IS A BUG

Sometimes the software does exactly what the spec says, and the user still had a bad experience. That's not a defect in the code, but it may be a problem in the requirements.

"Works as designed" closes the ticket. "Works as designed, but is the design good?" opens a better conversation with the product owner.

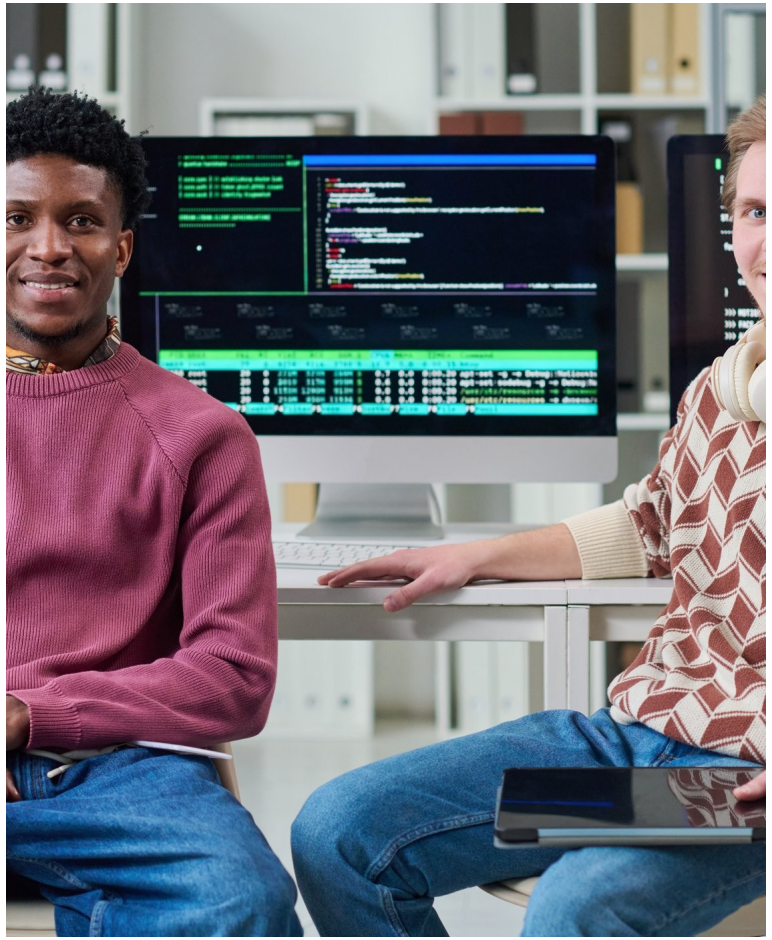


© 2026 by Innovation In Software Corporation

WORKING WITH DEVELOPERS

QA and development are on the same side: shipping software that works. The way you communicate a bug decides how quickly it gets fixed and how the next one is received.

- Bring **facts and evidence**, not opinions
- Make it easy: minimal steps, the exact request, the log line
- Ask questions instead of assigning blame
- Offer to help: pair on it, retest quickly, write the regression test



© 2026 by Innovation In Software Corporation

A GOOD BUG MESSAGE

"Search is broken again!! Your change crashed the server. Please fix ASAP."

A good message says:

- **What** happens: the symptom, in one sentence
- **How** to reproduce it: one or two steps
- **Evidence:** the status code, the log line, the stack trace location
- **Impact:** who is affected and how badly
- A **question**, not an accusation: "Could Friday's change be involved?"

© 2026 by Innovation In Software Corporation

BLAMELESS COMMUNICATION

Bugs come from systems: rushed deadlines, unclear requirements, missing tests. Blaming a person makes people defensive and hides the real cause.

- Describe the **behavior**, not the developer: "this change" rather than "your mistake"
- Assume good intent
- Focus on preventing the next one: which test would have caught it?

© 2026 by Innovation In Software Corporation

TALKING TO SUPPORT AND CUSTOMERS

Support teams and customers need a different message from developers: plain language, no jargon, and a clear next step.

- Say what happened in terms the customer understands
- Say what they can do now, such as a workaround or a wait time
- Say what happens next, and when they'll hear back
- Never share internal details like stack traces with customers

© 2026 by Innovation In Software Corporation

LAB 3.1 – REPRODUCE, ISOLATE, TRIAGE

It's Monday morning after a weekend beta, and support has forwarded six tickets. You're running triage.

- Investigate a "search does nothing" ticket with the Console, Network tab, server log, and a direct API call
- Read a stack trace and minimize the input that triggers the crash
- Reproduce, isolate, and classify five more tickets, with severity, priority, and outcome
- Rank the fix queue, write a message to the developer, and reply to support

© 2026 by Innovation In Software Corporation

POP QUIZ:

When reading a stack trace, which line should you look at first after the error message?

A. The first frame inside your own code

B. The last line of the trace

C. Any frame inside `node_modules`

D. The line with the longest file path





POP QUIZ:

When reading a stack trace, which line should you look at first after the error message?

A. The first frame inside your own code

B. The last line of the trace

C. Any frame inside `node_modules`

D. The line with the longest file path





POP QUIZ:

The company logo is misspelled on the home page during launch week. How would you rate it?

A. High severity, low priority

B. Low severity, high priority

C. High severity, high priority

D. Low severity, low priority



POP QUIZ:

The company logo is misspelled on the home page during launch week. How would you rate it?

A. High severity, low priority

B. Low severity, high priority

C. High severity, high priority

D. Low severity, low priority



POP QUIZ:

The API returns the correct cart total, but the page still shows the old one. Where is the bug?

A. In the back end

B. In the database

C. In the front end

D. In the requirements



POP QUIZ:

The API returns the correct cart total, but the page still shows the old one. Where is the bug?

A. In the back end

B. In the database

C. In the front end

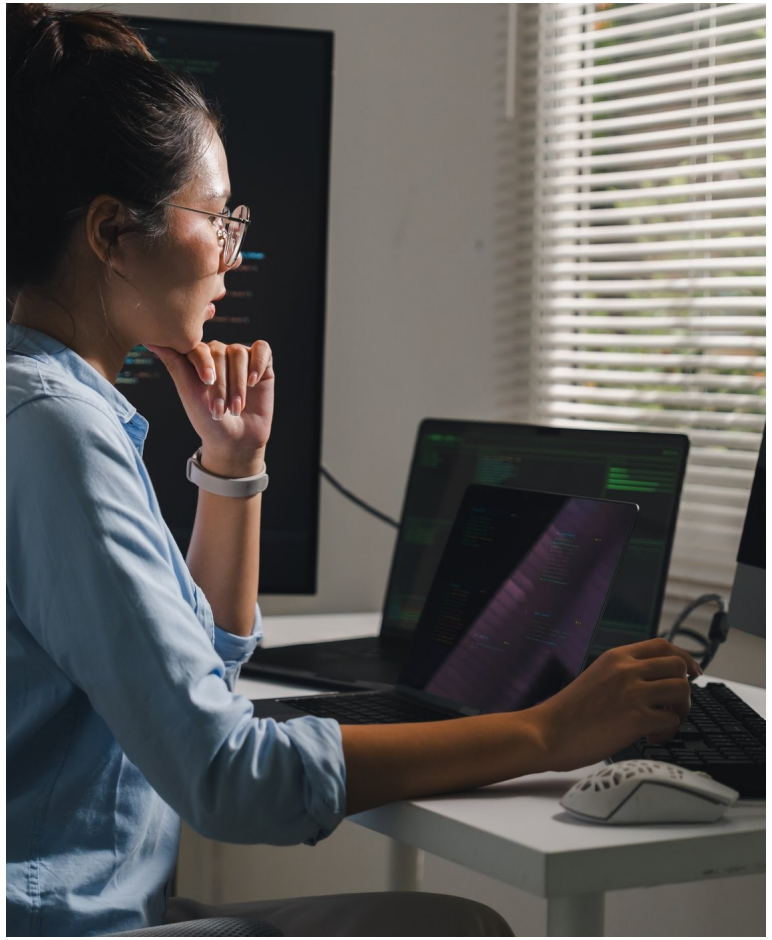
D. In the requirements



FROM TRIAGE TO TRACKER

Triage tells you what's real and how bad it is. The bug report is how that knowledge reaches the people who fix it, and the tracker is where the whole team sees what's open, who owns it, and what's next.

A bug that lives only in a chat message or someone's memory will be forgotten. A bug in the tracker gets fixed, verified, and counted.



© 2026 by Innovation In Software Corporation

WHAT MAKES A BUG REPORT ACTIONABLE?

Developers ask three questions about every bug. A good report answers all three before they have to ask.

- **Can I reproduce it?** Exact steps, data, and environment
- **Do I know what "right" looks like?** An expected result with its source
- **Is it worth fixing now?** Impact, severity, and priority

If a developer has to come back to you with questions, the report wasn't finished.

© 2026 by Innovation In Software Corporation

ANATOMY OF A BUG REPORT

Field	Purpose
Title	What's wrong, where, under what condition. Searchable.
Environment	Build, browser and version, OS, URL, test account
Preconditions	The state needed before step 1
Steps to reproduce	Numbered, minimal, one action per step
Expected result	What should happen, with its **source**
Actual result	What did happen: facts, no opinions
Evidence	Screenshot, network response, log lines, stack trace
Reproducibility	Always (5/5) or intermittent (2/10)
Severity and priority	Impact and urgency
Links	Support ticket, test case, related bugs

© 2026 by Innovation In Software Corporation

WRITING A GOOD TITLE

The title is the most-read part of a bug report. It shows up in lists, search results, and notifications. It should say **what's wrong**, **where**, and **when**.

Weak title	Strong title
Discount bug	Cart: second discount code is added instead of replacing the first
Login broken	Login: correct password rejected after a failed attempt on another account
Page error	Book details: page shows "Book not found" for IDs above 30

Ask yourself: would someone find this bug by searching for the obvious words?

© 2026 by Innovation In Software Corporation

STEPS TO REPRODUCE

Steps are a recipe. Someone who's never seen the bug should be able to follow them and see it happen.

- Start from a **known state**: "Empty cart, logged out"
- **One action per step**, numbered
- Include the **exact data**: the book, the code, the quantity
- Keep only the steps that matter: minimize first
- Note anything special about the environment

© 2026 by Innovation In Software Corporation

EXPECTED VS. ACTUAL

Expected says what should happen and **why** you think so: a requirement, an acceptance criterion, or a test case ID. Without a source, it's just your opinion against the developer's.

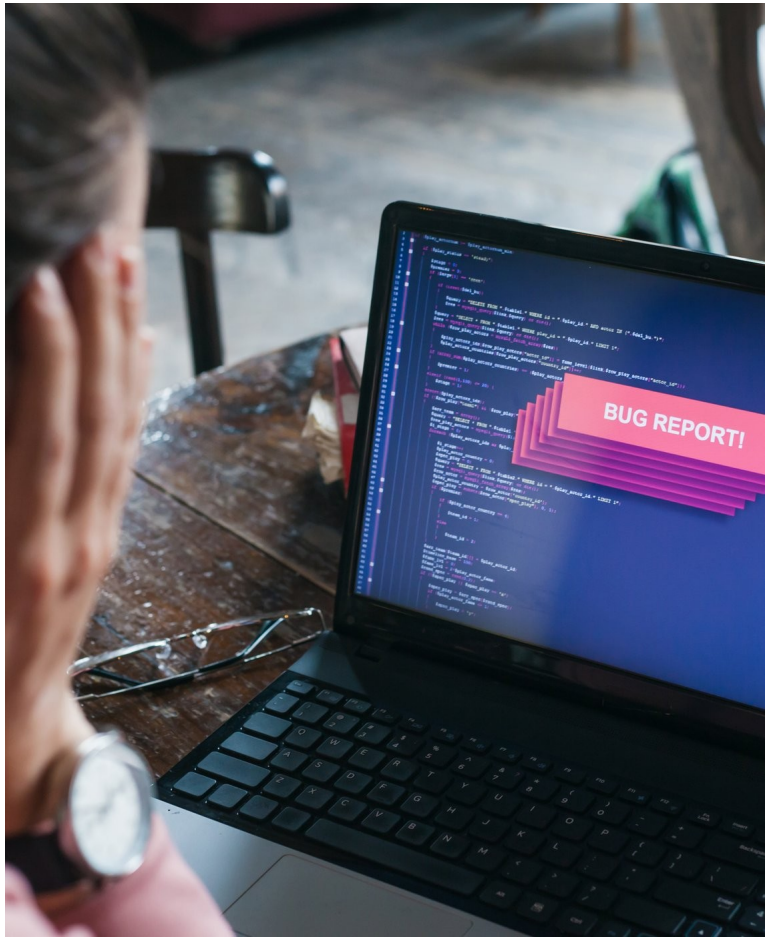
Actual says what did happen, in facts: the exact message, the exact amount, the status code. "It doesn't work" is not an actual result.

© 2026 by Innovation In Software Corporation

EVIDENCE: SHOW, DON'T TELL

Evidence turns "I saw a problem" into "here is the problem."

- **Screenshots** for what the user sees
- **Network responses** for what the API returned
- **Log lines and stack traces** pasted as **text**, so developers can search and copy them
- A short screen recording for problems that depend on timing or sequence



© 2026 by Innovation In Software Corporation

A BAD REPORT VS. A GOOD ONE

"Shipping is broken!! Fix please."

```
**Title:** Cart: $35.00 order gets free shipping (requirement says "over $35")
**Environment:** Local build, Chrome 141, Windows 11
**Preconditions:** Empty cart
**Steps:** 1. Add "Salt Fat Acid Heat" ($35.00)  2. Open the cart
**Expected:** Shipping $4.99 (F6.3: free shipping on orders over $35)
**Actual:** Shipping $0.00
**Evidence:** GET /api/cart returns "subtotal": 3500, "shipping": 0
**Reproducibility:** Always (3/3)
```

© 2026 by Innovation In Software Corporation

COMMON BUG REPORT MISTAKES

Mistake	Why it hurts
Vague title	Nobody can find it, and duplicates pile up
"Go to the site and add some books"	The developer can't reproduce your exact case
No expected result, or no source for it	It becomes an argument instead of a fix
Stack trace as a screenshot	Can't be searched or copied
Severity and priority missing, or always "High"	The team can't prioritize
Opinions and blame	People get defensive; the facts get lost

© 2026 by Innovation In Software Corporation

ONE BUG PER REPORT

Each report should describe **one** problem with **one** root cause. Two problems in one report means one gets fixed and the other gets forgotten when the ticket closes.

When one symptom has two causes in different layers, such as an API that crashes and a page that hides the error, consider two linked reports. They may have different fixes and different owners.

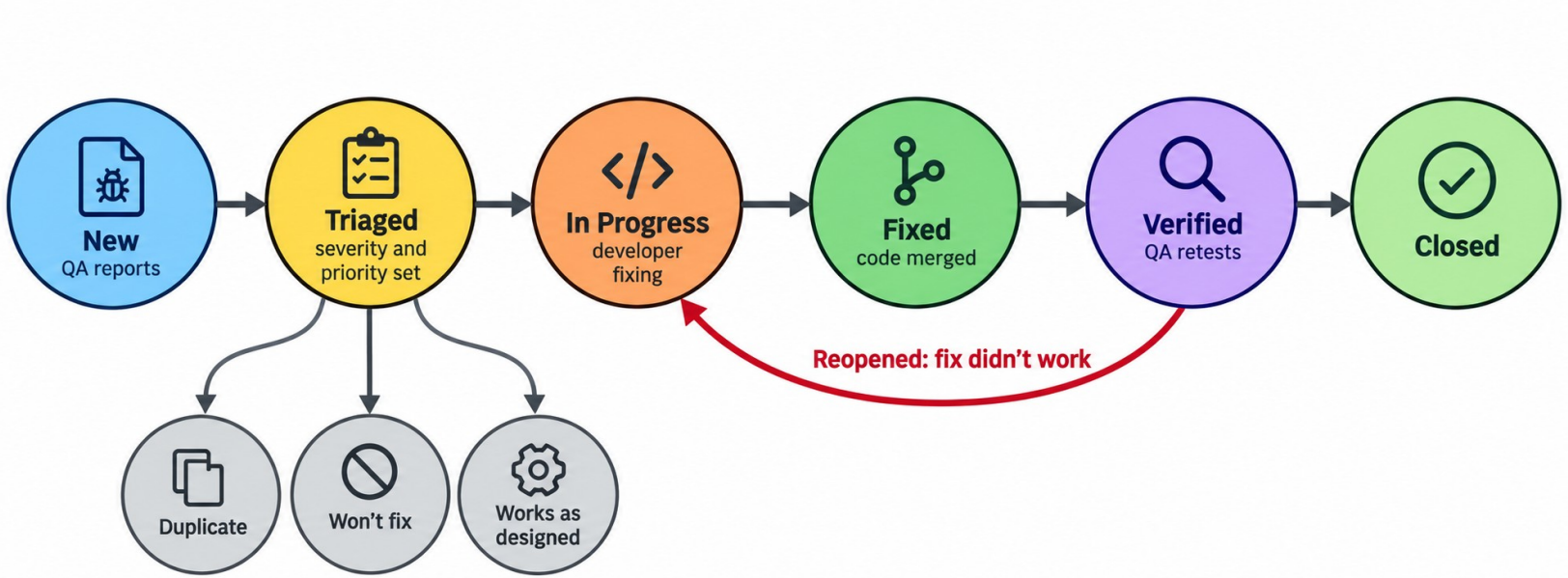
© 2026 by Innovation In Software Corporation

THE BUG LIFECYCLE

A bug moves through states from the moment it's reported until it's verified fixed. QA usually opens it and closes it.

© 2026 by Innovation In Software Corporation

THE BUG LIFECYCLE



© 2026 by Innovation In Software Corporation

ISSUE TRACKING TOOLS

Every team uses some kind of issue tracker. The tool matters less than how consistently the team uses it.

- **Jra:** the most widely used tracker in software teams
- **Azure DevOps Boards:** common in Microsoft-centered organizations
- **GitHub Issues and Projects:** built into the code repository
- Others: GitLab Issues, Linear, YouTrack, and many more

The concepts carry over: issues, types, fields, workflows, filters, and dashboards.



© 2026 by Innovation In Software Corporation

SAME IDEAS, DIFFERENT TOOLS

Concept	GitHub (the lab)	Jira	Azure DevOps
A bug	Issue with the `bug` label	Issue type **Bug**	Work item type **Bug**
Severity	Custom project field	Custom field (or labels)	Built-in **Severity** field
Priority	Custom project field	Built-in **Priority**	Built-in **Priority**
Saved search	Project **view** with a filter	**JQL** filter	**Query**
Dashboard	Project views and **Insights** charts	**Dashboard** with gadgets	**Dashboard** with widgets

© 2026 by Innovation In Software Corporation

JIRA

Jra organizes work into **projects** and **issues**. Each issue has a type, such as Story, Task, or Bug, and moves through a configurable workflow.

- **JQL** (Jra Query Language) builds saved filters: `project = BN AND type = Bug AND status != Done`
- **Dashboards** combine gadgets: filter results, pie charts, two-dimensional grids
- Rich integrations with test management tools and CI systems

© 2026 by Innovation In Software Corporation

AZURE DEVOPS

Azure DevOps Boards tracks **work items**: epics, user stories, tasks, and bugs, all linked together. It's part of a suite that also includes repositories, pipelines, and test plans.

- Bugs have **Severity** and **Priority** fields built in
- **Queries** filter work items and can be charted
- **Test Plans** link test cases, test runs, and bugs in one place

© 2026 by Innovation In Software Corporation

GITHUB ISSUES AND PROJECTS

GitHub Issues lives right next to the code. A bug can link directly to the commit that fixed it and the pipeline run that caught it.

- **Labels** and **issue templates** keep reports consistent
- **Projects** add custom fields, such as Severity and Priority, plus table and board views
- **Insights** turns project fields into charts
- One account for code, issues, and CI



© 2026 by Innovation In Software Corporation

FIELDS, LABELS, AND LINKS

A tracker is only as useful as the data in it. Consistent fields make bugs searchable, countable, and chartable.

- Use **dedicated fields** for severity and priority, not free text
- Use labels sparingly, for things like area or type
- **Link** everything: the support ticket, the test case, the failing pipeline run, related bugs
- Links turn a single bug into a trail anyone can follow

© 2026 by Innovation In Software Corporation

TEMPLATES KEEP REPORTS CONSISTENT

A template turns "write a bug report" into "fill in these blanks." Every report then has the same fields in the same order.

```
**Title:** <Area>: <what's wrong> <when / with what input>
**Environment:**
**Preconditions:**
**Steps to reproduce:**
**Expected result:** (source: F#.# / AC / TC-#)
**Actual result:**
**Evidence:**
**Reproducibility:**
**Links:**
```

Most trackers can apply a template automatically to every new bug.

© 2026 by Innovation In Software Corporation

DASHBOARDS VS. TEST REPORTS

	Dashboard	Test summary report
Time	**Live**, always current	A **snapshot** at a milestone
Audience	The team, every day	Stakeholders and decision makers
Answers	What's open, how bad, who's on it?	Did we meet the exit criteria? Should we ship?
Ends with	Nothing; it keeps updating	A **recommendation**

© 2026 by Innovation In Software Corporation

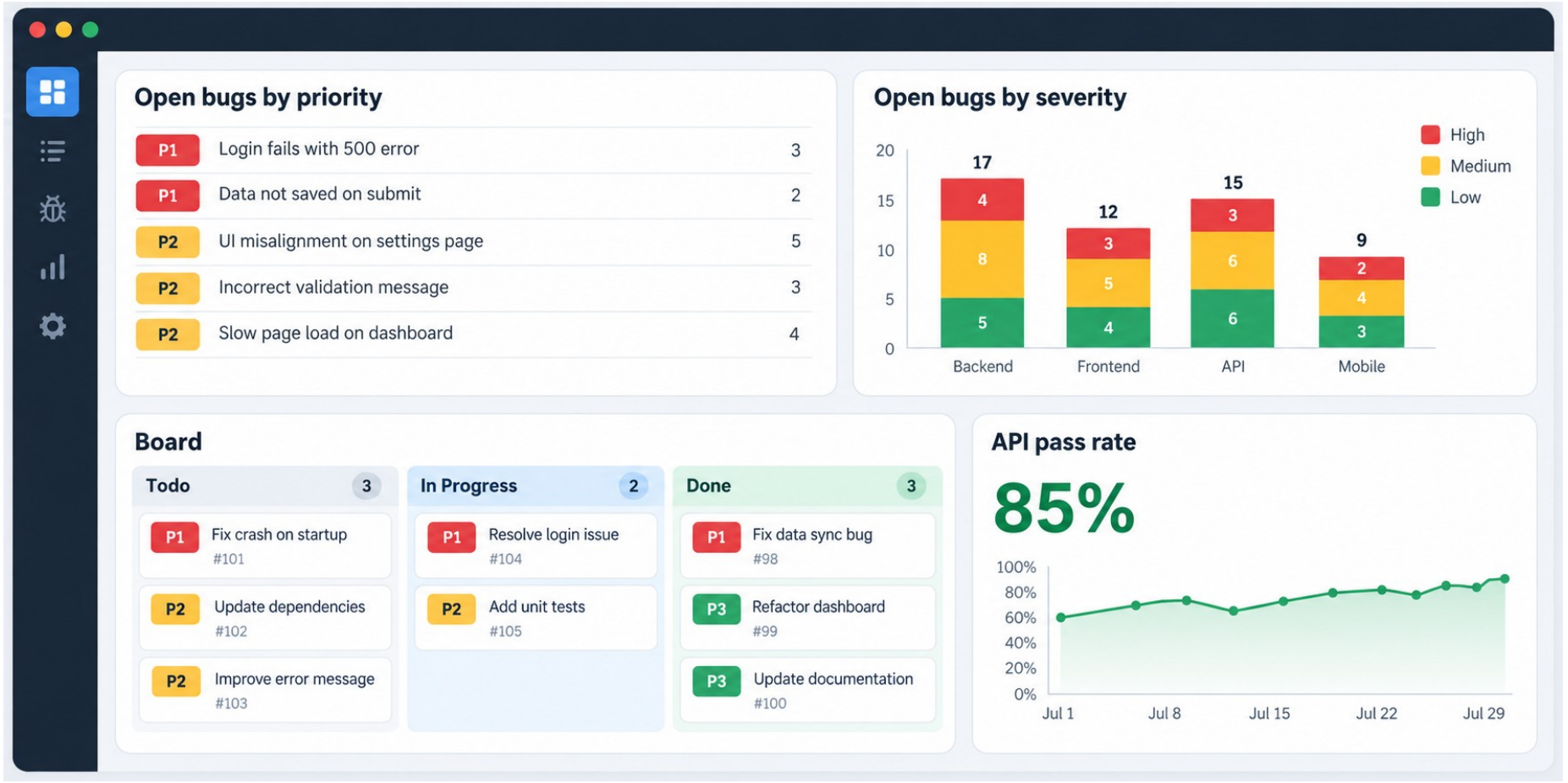
BUILDING A USEFUL DASHBOARD

A good dashboard answers the team's daily questions at a glance. Keep it to a few views that drive action.

- **Open bugs**, sorted by priority
- **Open bugs by severity**, to show how bad things are
- **Severity × priority**, to spot mismatches
- **Work in progress**, to see who's on what

© 2026 by Innovation In Software Corporation

BUILDING A USEFUL DASHBOARD



© 2026 by Innovation In Software Corporation

WHY MEASURE QUALITY?

"Is it ready?" deserves a better answer than "I think so." Metrics turn testing into evidence that stakeholders can weigh, compare over time, and act on.

Good metrics show where the risk is, whether things are improving, and whether the team met the bar it set for itself. They support judgment; they don't replace it.



© 2026 by Innovation In Software Corporation

TEST EXECUTION METRICS

Metric	Formula	Tells you
Execution rate	tests run ÷ tests planned	How much of the plan you actually did
Pass rate	tests passed ÷ tests run	How healthy the build looks under test
Pass rate by suite	per suite or per feature	**Where** the problems are

Example: 100 tests planned, 90 run, 81 passed. Execution rate 90%, pass rate 90%. Ten tests were never run, and that's a risk too.

© 2026 by Innovation In Software Corporation

DEFECT METRICS

Metric	Formula	Tells you
Open defects by severity	Count per level	How bad the open problems are
Defect density	Defects ÷ size (per feature, or per 1,000 lines of code)	Where bugs cluster
Defect detection percentage (DDP)	Found by QA ÷ (found by QA + found by customers)	How well testing catches bugs before users do
Reopen rate	Reopened ÷ fixed	How reliable fixes are

© 2026 by Innovation In Software Corporation

DEFECT DENSITY

Defect density shows where bugs cluster, so you know where to look harder next time.

- **Per feature:** "Discounts has 5 defects, Search has 2." This is easy to read and act on
- **Per 1,000 lines of code (KLOC):** useful for comparing releases of the same product
- Small codebases and planted bugs inflate it, so compare like with like
- Remember the testing principle: defects cluster

© 2026 by Innovation In Software Corporation

DEFECT DETECTION PERCENTAGE

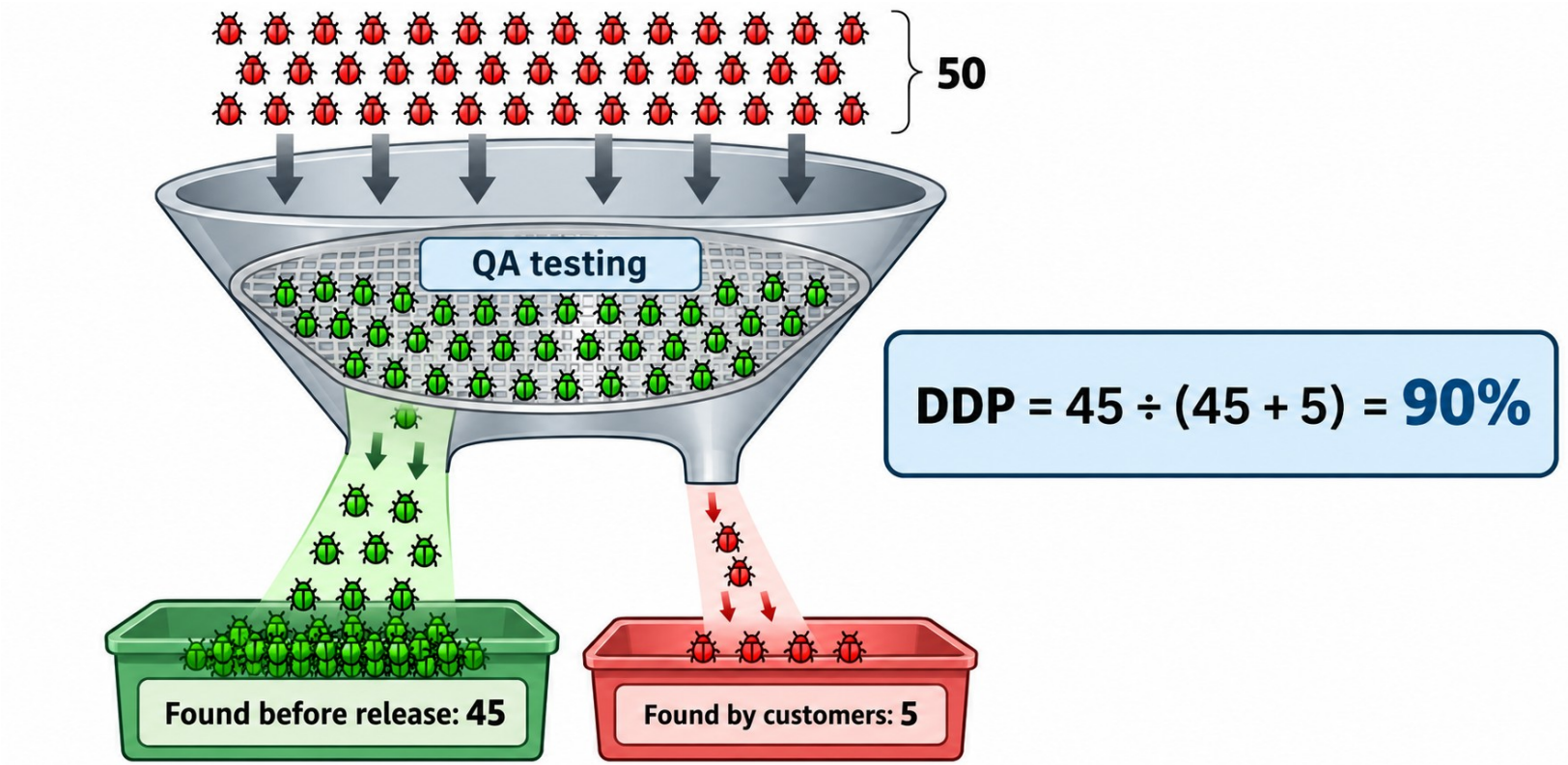
DDP asks: of all the bugs we know about, how many did **we** find before customers did?

If QA found 45 bugs before release and customers reported 5 after, DDP is 45 ÷ 50 = **90%.**

- Higher is better, but no team reaches 100%
- **Which** bugs escaped matters as much as how many: one escaped checkout crash outweighs ten typos

© 2026 by Innovation In Software Corporation

DEFECT DETECTION PERCENTAGE

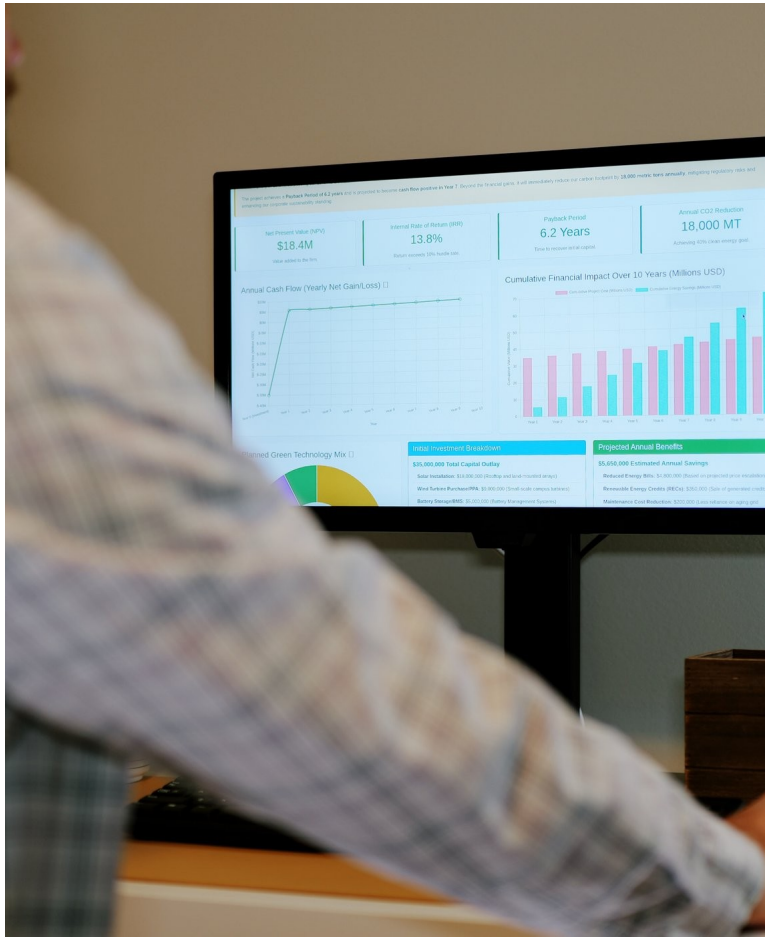


© 2026 by Innovation In Software Corporation

METRICS NEED CONTEXT

No single number tells the whole story. A 95% pass rate means little if the 5% that failed is checkout. 100% of unit tests passing says nothing about the UI.

- Report several metrics together
- Break them down by suite, feature, and severity
- Show trends, not just today's number
- Explain what the numbers mean and what they don't



© 2026 by Innovation In Software Corporation

WHEN METRICS BECOME TARGETS

When a metric becomes a target, people find ways to hit the target without improving quality.

Target	How it gets gamed
"Raise the pass rate"	Delete or skip failing tests
"Fewer open bugs"	Close bugs as "won't fix," or merge several into one
"More bugs found"	Report every tiny issue separately
"100% coverage"	Tests that run code but check nothing

Pair every metric with review, and watch what happens to the tests behind it.

© 2026 by Innovation In Software Corporation

THE TEST SUMMARY REPORT

A test summary report is written at a milestone, such as a release candidate, and ends with a recommendation.

- **Scope:** what was tested, and what wasn't and why
- **Results:** execution and pass rates, by suite
- **Defects:** open by severity, density, escapes
- **Exit criteria:** each one met, not met, or unknown
- **Risks** and a **recommendation:** go, no-go, or go with conditions

© 2026 by Innovation In Software Corporation

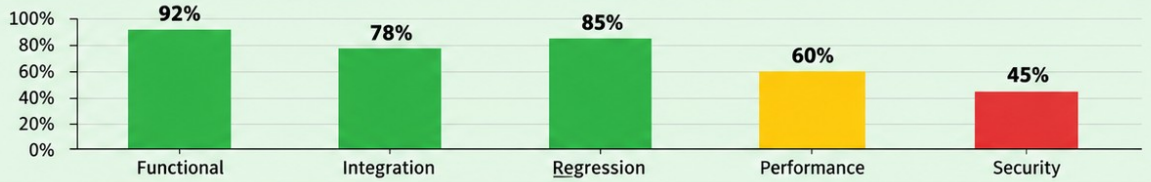
THE TEST SUMMARY REPORT

1. Scope

Test execution for Release 1.0 across the following test suites:

- Functional Testing
- Integration Testing
- Regression Testing
- Performance Testing
- Security Testing

2. Results



3. Defects

Severity	Open	Resolved	Total
Critical	3	1	4
High	7	12	19
Medium	12	28	40
Low	8	26	34

Total Defects
97

Open Defects
30

4. Exit Criteria

✓	All planned test cases executed (≥ 95%)
✓	No open critical or high severity defects
✗	Pass rate ≥ 80% for all test suites
✓	Performance meets agreed thresholds
✗	Security testing passed with no critical issues

5. Recommendation

NO-GO

because open critical defects remain.

© 2026 by Innovation In Software Corporation

EXIT CRITERIA AND THE RELEASE DECISION

The exit criteria you wrote on Day 1 come back here. Each one is checked against the numbers: met, not met, or can't tell.

"Go with conditions" is a legitimate answer, but only when it names which known problems are accepted, who signed off, and what happens next. Anything vaguer is a guess dressed up as a decision.



© 2026 by Innovation In Software Corporation

LAB 3.2 – BUG REPORTS & QUALITY DASHBOARD

In this lab you turn Monday's triage into actionable bug reports, build a quality dashboard, and answer the release manager's question: is it ready to ship?

- Set up GitHub Issues with a bug template and a project with Severity and Priority fields
- Write a full report for the new search crash, then for three customer-reported bugs
- Build dashboard views and a severity-by-priority chart
- Calculate quality metrics from release-candidate data and write a test summary report with a recommendation

© 2026 by Innovation In Software Corporation

POP QUIZ:

Which bug title is the most actionable?

A. Discount bug

B. Cart is broken!!

C. Cart: second discount code is added instead of replacing the first

D. Problem found during testing



POP QUIZ:

Which bug title is the most actionable?

A. Discount bug

B. Cart is broken!!

C. Cart: second discount code is added instead of replacing the first

D. Problem found during testing



POP QUIZ:

A team planned 100 tests, ran 90, and 81 passed. What is the pass rate?

A. 81 %

B. 90%

C. 100%

D. 72%



POP QUIZ:

A team planned 100 tests, ran 90, and 81 passed. What is the pass rate?

A. 81 %

B. 90%

C. 100%

D. 72%



POP QUIZ:

QA found 45 bugs before release, and customers reported 5 more afterward. What is the defect detection percentage?

A. 90%

B. 50%

C. 10%

D. 95%



POP QUIZ:

QA found 45 bugs before release, and customers reported 5 more afterward. What is the defect detection percentage?

A. 90%

B. 50%

C. 10%

D. 95%



EVERY CHANGE, EVERY TIME

So far, every test you've written runs only when someone remembers to run it. A teammate "cleans up" some code on Friday, nobody runs the tests, and on Monday customers find the problem.

Continuous integration ends that. Every change runs the tests automatically, on a clean machine, and the team knows within minutes whether something broke and which change did it.



© 2026 by Innovation In Software Corporation

WHAT IS CONTINUOUS INTEGRATION?

Continuous integration (CI) means developers merge small changes often, at least daily, and every merge is automatically built and tested.

- Small changes are easy to review, test, and undo
- Problems are found minutes after they're introduced, by the person who introduced them
- The main branch always works, or the team knows immediately that it doesn't



© 2026 by Innovation In Software Corporation

CONTINUOUS DELIVERY AND DEPLOYMENT

Practice	What happens automatically	Release to users
Continuous integration	Every change is built and tested	Separate decision
Continuous delivery	Every passing change is ready to release	A person presses the button
Continuous deployment	Every passing change goes to production	Automatic

Each step to the right puts more trust in the automated tests. That trust has to be earned, and QA is the one who earns it.

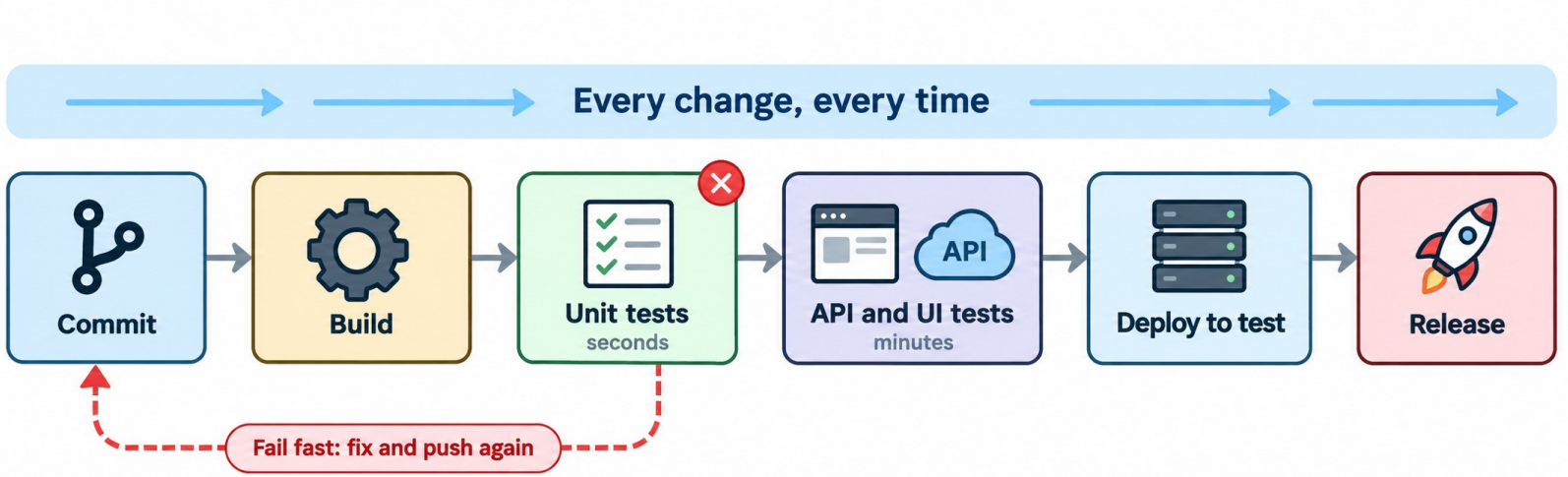
© 2026 by Innovation In Software Corporation

THE CI/CD PIPELINE

A pipeline is the automated path every change follows, from commit to release. Each stage must pass before the next one runs.

© 2026 by Innovation In Software Corporation

THE CI/CD PIPELINE



© 2026 by Innovation In Software Corporation

WITHOUT CI VS. WITH CI

Without CI	With CI
Tests run when someone remembers	Tests run on **every** push and pull request
Bugs found days later, cause unclear	Bugs found in minutes, tied to **one** change
QA tests at the end, as a phase	QA designs the checks; the pipeline runs them all the time
"Works on my machine"	Runs on a clean machine every time
Big, risky merges	Small, frequent, low-risk merges

© 2026 by Innovation In Software Corporation

THE ROLE OF QA IN CI/CD

In a CI/CD team, QA spends less time **running** tests and more time **designing** the safety net. The pipeline runs the checks; QA decides what they are.

- Which tests run, and in what order
- Which failures block a merge, and which are only reported
- What the feedback says when something fails
- Whether the pipeline can be trusted



© 2026 by Innovation In Software Corporation

CONTINUOUS TESTING

Continuous testing means testing runs throughout the delivery process, not in one phase near the end. Every stage gives feedback about risk.

- Unit tests on every commit
- API and UI tests on every pull request
- Slower suites, like visual, cross-browser, and load tests, nightly or before release
- Monitoring in production after release

Testing becomes a constant signal, not a gate at the end.

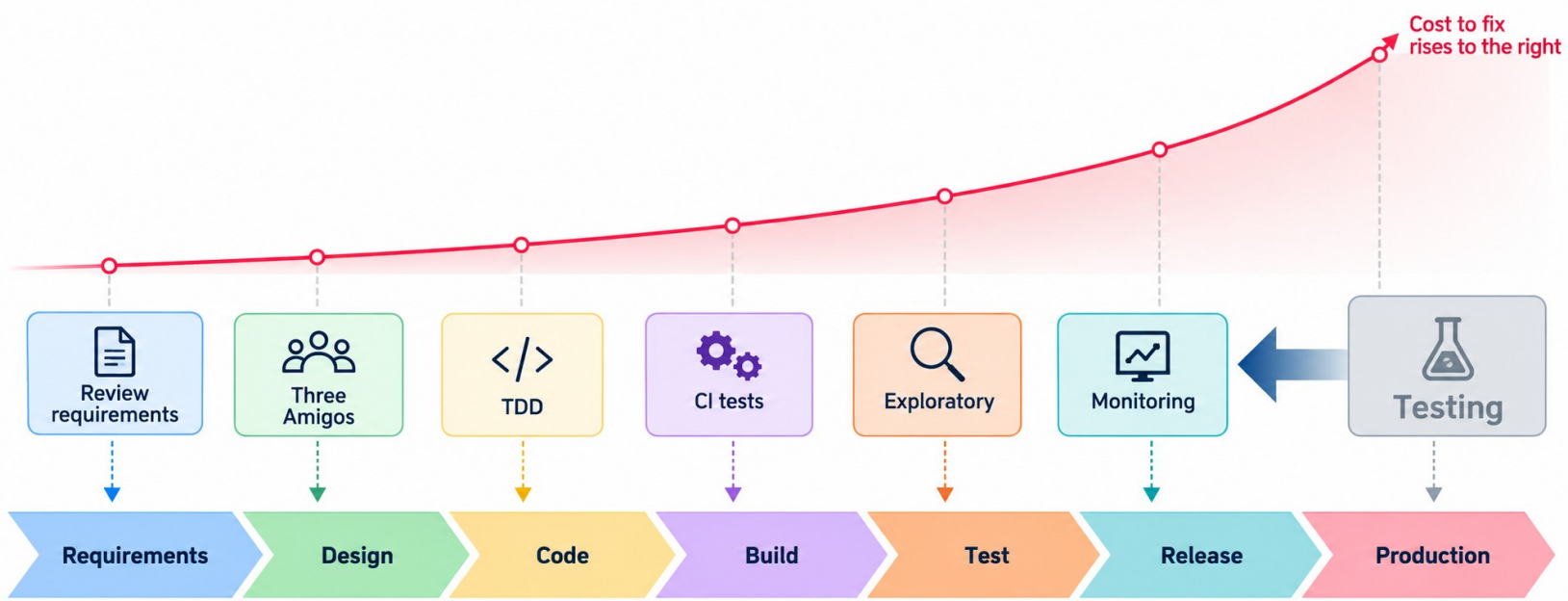
© 2026 by Innovation In Software Corporation

SHIFT-LEFT TESTING

Shift left means moving testing earlier in the lifecycle, toward the left of the timeline, where defects are cheaper to fix.

© 2026 by Innovation In Software Corporation

SHIFT-LEFT TESTING



© 2026 by Innovation In Software Corporation

SHIFT-LEFT IN PRACTICE

You've already done shift-left testing this week.

Activity	Shifted to	Lab
Find vague requirements	Before any code exists	1.1
Write acceptance criteria and test cases	Before the sprint starts	1.3
Write the test before the code	During development	2.1
Test the API before the UI exists	Using a mock	2.4
Run every test on every change	Before the merge	Today

© 2026 by Innovation In Software Corporation

SHIFT RIGHT, TOO

Some problems only appear with real users, real data, and real traffic. **Shift right** extends testing into production.

- Monitor errors, response times, and logs after each release
- Release to a small group first, then widen
- Feed production problems back into new tests

The best teams do both: catch what they can early, and learn fast from what escapes.

© 2026 by Innovation In Software Corporation

THE FEEDBACK LOOP

Feedback is only useful if it's fast and specific. A failure an hour later, tied to one change, is a five-minute fix. A failure a week later, after fifty changes, is an investigation.

Feedback source	Typical time	Cost to act on it
Unit tests in CI	Seconds	Tiny
API and UI tests in CI	Minutes	Small
Nightly suites	Hours	Moderate
Manual regression	Days	Large
Customer report	Weeks	Largest

© 2026 by Innovation In Software Corporation

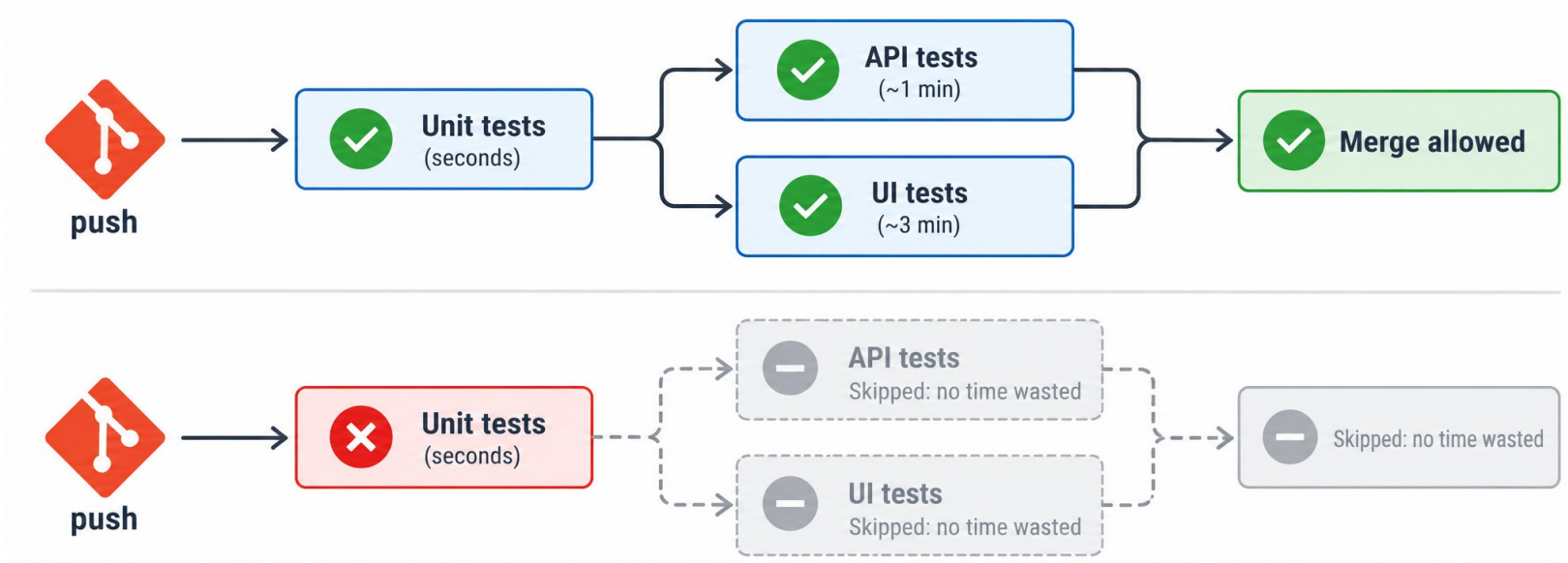
ORDER STAGES FOR FAST FEEDBACK

Put the **fastest, most precise** tests first. If they fail, don't spend minutes running slower ones.

- Unit tests first: seconds, and they point to one function
- API and UI tests next, in **parallel**, only if unit tests pass
- Slow suites last, or on a schedule

© 2026 by Innovation In Software Corporation

ORDER STAGES FOR FAST FEEDBACK



© 2026 by Innovation In Software Corporation

GITHUB ACTIONS

GitHub Actions is the CI/CD service built into GitHub. Pipelines are YAML files that live in the repository, next to the code and tests they run.

- Free for public repositories, with included minutes for private ones
- Runs on GitHub-hosted Linux, Windows, and macOS machines
- Results appear right on commits and pull requests
- Other CI tools, such as Jenkins, GitLab CI, and Azure Pipelines, use the same ideas



© 2026 by Innovation In Software Corporation

PIPELINE VOCABULARY

Term	Meaning	In the lab
Workflow	A YAML file in `.github/workflows/`	`qa-pipeline.yml`
Trigger (`on:`)	What starts it	Push to `main`, any pull request, a manual run
Job	A group of steps on one fresh machine (a **runner**)	`unit`, `api`, `ui`
Step	One command or action	`npm test`
`needs:`	Job order: wait for another job to pass	`api` and `ui` need `unit`
Status check	A job's pass/fail result on a pull request	or next to each job

© 2026 by Innovation In Software Corporation

ANATOMY OF A WORKFLOW

```yaml
name: QA Pipeline

on:
  push:
    branches: [main]
  pull_request:

jobs:
  unit:
    name: Unit tests
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-node@v7
        with:
          node-version: 22
      - run: npm install
      - run: npm test
```

Check out the code, set up the tools, install, test. Almost every test job follows this shape.

© 2026 by Innovation In Software Corporation

A FRESH MACHINE EVERY TIME

Every job starts on a brand-new runner with nothing installed. That's what makes CI trustworthy: no leftover files, no "works on my machine."

- Each job installs its own tools and dependencies
- Each job starts its own copy of the app if it needs one
- Anything a job needs from another must be passed explicitly
- If it passes on the runner, it passes for a reason

© 2026 by Innovation In Software Corporation

RUNNING THE APP UNDER TEST

API and UI tests need the application running on the runner. Start it in the background, then **wait until it answers** before testing.

```
- name: Start BookNook
  working-directory: sample-app
  run: |
    npm ci
    node server.js &
    timeout 30 bash -c 'until curl -s http://localhost:3000 > /dev/null; do sleep 1; done'
```

Without the wait, the first tests can run before the server is up and fail at random.

© 2026 by Innovation In Software Corporation

INTEGRATING EACH KIND OF TEST

Test type	Tool	Runs in CI with
Unit	pytest or Jest	`python -m pytest` or `npm test`
API	Postman	`npx newman run collection.json -e environment.json`
UI	Cypress	`npx cypress run --browser chrome --spec ...`

Every tool you've used this week runs **headless** from the command line, and every one returns a non-zero exit code on failure. That exit code is what turns a step red.

© 2026 by Innovation In Software Corporation

WHAT NOT TO RUN ON EVERY CHANGE

Not every test belongs on every pull request. Some are slow, some need special setup, and some can't be stable on a shared runner.

Suite	Why not every change	Run instead
Visual regression	Baselines must be made in the CI environment	Nightly, with CI-made baselines
Full cross-browser	Slow, needs many browsers	Nightly or before release
Load and performance	Needs a production-like environment	Before release
Full accessibility audit	Slow; includes manual checks	Per sprint or before release

© 2026 by Innovation In Software Corporation

GATE VS. REPORT

	Gate (blocking)	Report (non-blocking)
A failure...	Turns the build red and blocks the merge	Is shown, but the build stays green
Use for	Tests that must pass on every change	Known, **tracked** defects; checks still being tuned
Risk	A gate that's always red gets ignored	A report nobody reads

The pipeline's job is to say "this change broke something." A gate that's red for reasons unrelated to the change teaches people to ignore red.

© 2026 by Innovation In Software Corporation

KNOWN FAILURES AND QUARANTINE

Sometimes tests fail because of known bugs that are tracked but not fixed yet. Running them as a gate makes every build red; deleting them loses the coverage.

Quarantine them: keep them running, report the results, link each one to its bug, and move it back to the gate in the same pull request that fixes the bug.

- Every report-only test has an issue link
- The team reviews the quarantine list regularly
- Quarantine is temporary, or it becomes "ignored forever"



© 2026 by Innovation In Software Corporation

THE RATCHET

"Report only" has a weakness: a **new** failure can slip through alongside the known ones. A **ratchet** fixes that.

- Record today's number of known failures
- Fail the build if the count goes **up**
- Lower the limit each time a bug is fixed

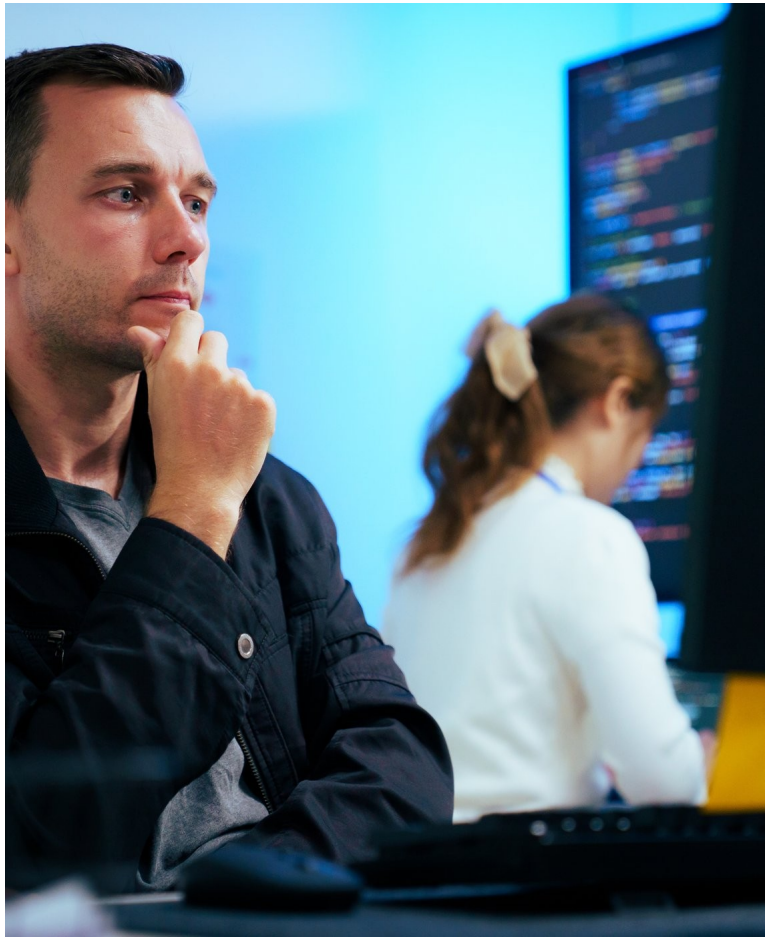
Known problems are allowed; new ones aren't. The number can only move in one direction.

© 2026 by Innovation In Software Corporation

STATUS CHECKS AND PROTECTED BRANCHES

A pipeline that nobody is required to pass is only a suggestion. **Required status checks** make it a rule: a pull request can't merge until the chosen jobs pass.

- Protect the main branch with a rule or ruleset
- Choose which checks are required
- "Please run the tests" becomes "it can't merge until they pass"



© 2026 by Innovation In Software Corporation

READING PIPELINE FEEDBACK

When a check fails, read it the way you read a stack trace: from the outside in.

- **Which job** failed, and which ones were skipped because of it?
- **Which step** failed inside that job?
- **Which test** failed: its name, ideally with a test case ID
- **Expected vs. actual**: the assertion message
- **Which change** caused it: the commit or pull request

Good test names make this fast. `TC-3.1 percent code takes percentage off` tells you the rule before you open the log.

© 2026 by Innovation In Software Corporation

ARTIFACTS AND SUMMARIES

A pipeline can leave behind more than a red or green mark.

- **Artifacts:** files saved from the run, such as failure screenshots and test reports, downloadable from the run page
- **Summaries:** short results written to the run's summary page, such as a pass rate
- **Links:** paste the run URL into a bug report as evidence

The metrics from Lab 3.2 become live numbers that update on every push.

© 2026 by Innovation In Software Corporation

KEEPING PIPELINES FAST

A slow pipeline gets skipped, batched, or ignored. Speed is part of quality.

Technique	Effect
Run jobs in **parallel**	Total time is the slowest job, not the sum
Cache dependencies	Skip reinstalling unchanged packages
Fail fast with `needs:`	Don't run slow stages after a fast one fails
Push checks down the pyramid	More unit and API tests, fewer UI tests
Move slow suites to a schedule	Nightly instead of every pull request

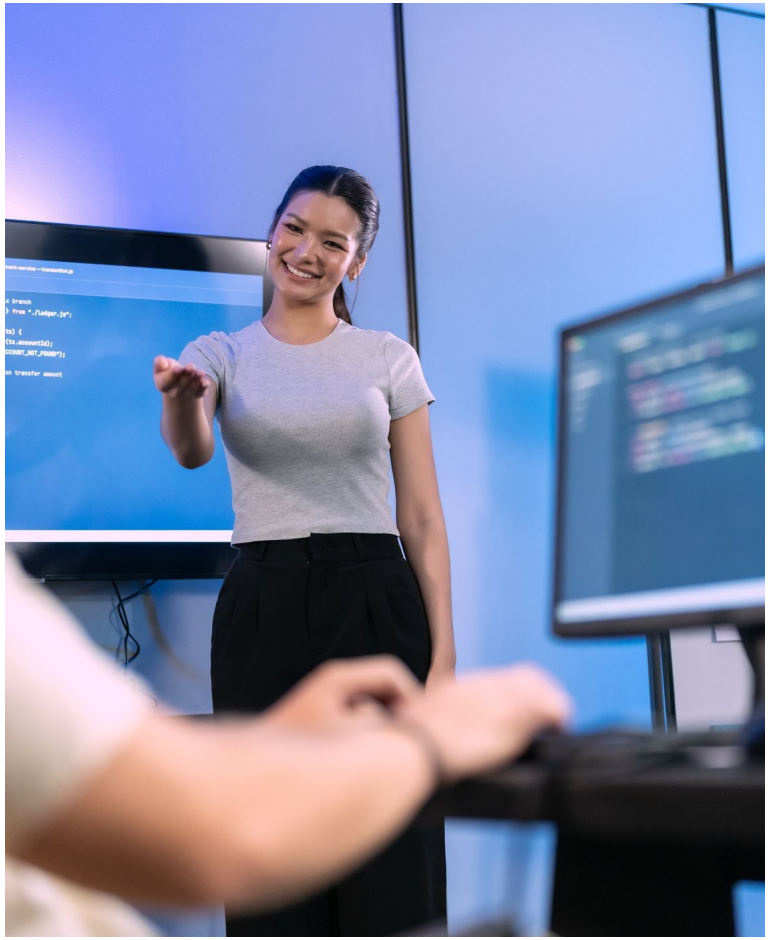
© 2026 by Innovation In Software Corporation

A RED PIPELINE MUST MEAN SOMETHING

The pipeline is only as valuable as the team's trust in it. If red often means "flaky" or "known issue," people stop looking, and real failures slip through.

- Gate only on tests that must pass
- Fix or quarantine flaky tests quickly
- Treat a red main branch as the team's top priority

Tomorrow's problem is today's ignored failure.



© 2026 by Innovation In Software Corporation

LAB 3.3 – TESTS IN A CI PIPELINE

In this lab you build a GitHub Actions pipeline for your course work, then face a teammate's pull request that breaks your code.

- Push your work to GitHub and add a unit test stage
- Add API (Newman) and UI (Cypress) stages that start BookNook on the runner
- Split the suites into gates and reports, and publish a pass rate summary
- Protect `main`, review a teammate's failing pull request, and fix it before merging

© 2026 by Innovation In Software Corporation

POP QUIZ:

What does "shift left" mean in testing?

A. Moving testing to the end of the project

B. Moving testing earlier in the lifecycle, where defects are cheaper to fix

C. Running tests only on the left side of the pipeline graph

D. Letting developers skip testing



POP QUIZ:

What does "shift left" mean in testing?

A. Moving testing to the end of the project

B. Moving testing earlier in the lifecycle, where defects are cheaper to fix

C. Running tests only on the left side of the pipeline graph

D. Letting developers skip testing





POP QUIZ:

Why does a pipeline usually run unit tests before API and UI tests?

A. Unit tests are more important than other tests

B. GitHub Actions requires it

C. UI tests can't run until unit tests create the database

D. They're the fastest and most precise, so a failure stops the pipeline early





POP QUIZ:

Why does a pipeline usually run unit tests before API and UI tests?

A. Unit tests are more important than other tests

B. GitHub Actions requires it

C. UI tests can't run until unit tests create the database

D. They're the fastest and most precise, so a failure stops the pipeline early





POP QUIZ:

Several API tests fail on every run because of known, tracked bugs. What's the best way to handle them in the pipeline?

A. Delete them until the bugs are fixed

B. Keep them as a blocking gate so every build is red

C. Run them as report-only, link each to its bug, and move them back to the gate when fixed

D. Re-run the pipeline until they pass



POP QUIZ:

Several API tests fail on every run because of known, tracked bugs. What's the best way to handle them in the pipeline?

A. Delete them until the bugs are fixed

B. Keep them as a blocking gate so every build is red

C. Run them as report-only, link each to its bug, and move them back to the gate when fixed

D. Re-run the pipeline until they pass



WHAT IS AN ANTIPATTERN?

An **antipattern** is a common response to a problem that seems reasonable at first but makes things worse over time. "Just re-run it." "Add a longer wait." "We'll test it by hand before release."

Every QA team falls into some of these. The goal isn't to never make mistakes; it's to recognize the pattern early and know the better alternative.



© 2026 by Innovation In Software Corporation

COMMON TESTING PITFALLS

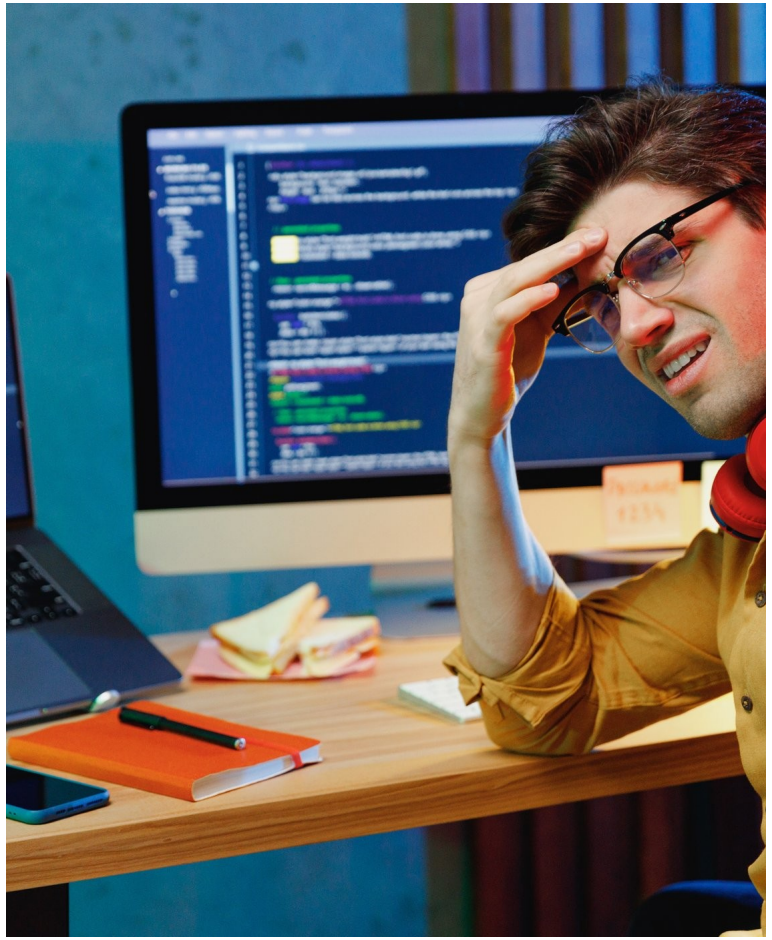
Pitfall	Looks like	Better practice
Testing everything equally	No priorities, effort spread thin	Risk-based testing
Only the happy path	Bugs found by customers at the edges	Negative, boundary, and combination tests
Vague expected results	"An error is shown" passes for anything	Exact messages and values
Tests that depend on each other	Pass alone, fail together	Each test owns its setup
Ignoring red builds	"That one always fails"	A red pipeline must mean something
Testing only at the end	Bugs found late and expensive	Shift left

© 2026 by Innovation In Software Corporation

WHAT IS A FLAKY TEST?

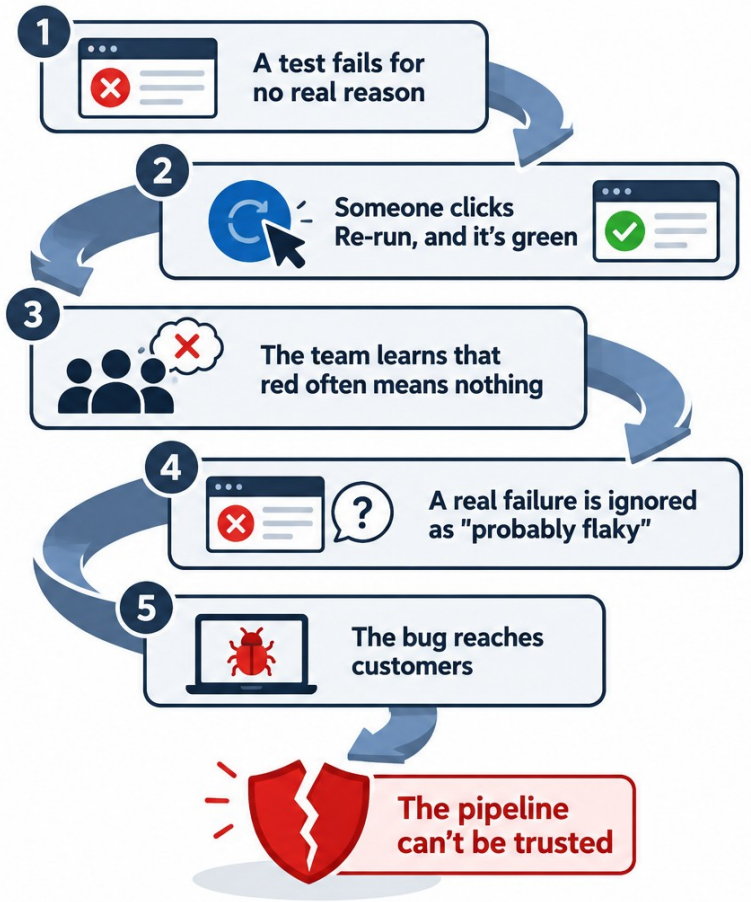
A **flaky test** passes and fails on the **same code**, with no change at all. Run it ten times and you might get eight greens and two reds.

The cause is almost never random. Something the test depends on varies between runs: timing, order, data, or what another test left behind. The test is checking more than it should, and controlling less.



© 2026 by Innovation In Software Corporation

WHY FLAKY TESTS ARE SO COSTLY



© 2026 by Innovation In Software Corporation

THE MATH OF FLAKINESS

A test that fails once in 50 runs sounds nearly reliable. It isn't.

Pipeline runs per day	Chance of at least one false red per day
1	2%
10	About 18%
30	About 45%

Now multiply by ten such tests. Nearly every day has a false failure, and the team stops believing red.

© 2026 by Innovation In Software Corporation

FIVE COMMON CAUSES

Antipattern	Looks like	Why it's flaky	Reliable pattern
Hard-coded wait	`wait(500)`, then a check	Fast machine passes; slow machine fails	Wait for a **condition** or a request
Fragile selector	Position and structure in the selector	Breaks when the layout changes	Select by meaning: test attributes, visible text
Order dependence	Test B uses what test A set	Fails alone, reordered, or in parallel	Each test sets up what it needs
Random data	A random pick each run	Fails only when the pick hits a special case	Fixed, chosen data
Shared state	No reset between tests	Fails after another test changes the system	Reset before each test

© 2026 by Innovation In Software Corporation

HARD-CODED WAITS

A fixed wait is a guess about how long something takes. On a fast laptop the guess is right; on a busy CI runner it's wrong.

```
//    Guess, then check once
cy.get('#login-btn').click();
cy.wait(1000);
cy.get('#message').then($m => expect($m.text()).to.eq('Welcome back'));

//    Check until it's true (up to the timeout)
cy.get('#login-btn').click();
cy.get('#message').should('have.text', 'Welcome back');
```

Too short: flaky. Long enough: every run pays the full wait, even when the page is ready in 50 ms.

© 2026 by Innovation In Software Corporation

WAIT FOR CONDITIONS, NOT TIME

Reliable tests wait for **evidence** that the app is ready, not for a number of milliseconds.

- **Retrying assertions:** `.should()` rechecks until the condition is true or the timeout expires
- **Network waits:** intercept a request, then wait for it to finish
- **Ready signals:** wait for a key element or text that appears only when loading is done
- A check that runs **once** (like `.then()` with `expect`) doesn't retry

© 2026 by Innovation In Software Corporation

FRAGILE SELECTORS

A selector tied to page structure breaks when anything around it changes, even when the feature still works perfectly.

- `#results > div:nth-child(6) > button` depends on order and nesting
- `.btn-primary` depends on styling
- A test that passes but doesn't check **which** item it clicked is testing luck

Prefer test attributes and visible text. When a designer wraps a button in a new container, a good test shouldn't notice.

© 2026 by Innovation In Software Corporation

ORDER DEPENDENCE

Tests that pass only in a certain order are a trap. Run them alone, in a different order, or in parallel, and they fail.

The usual cause: one test stores something, such as a variable, a created record, or a logged-in session, and another test uses it.

- Each test should set up exactly what it needs
- No shared variables between tests
- Every test should pass when run on its own

© 2026 by Innovation In Software Corporation

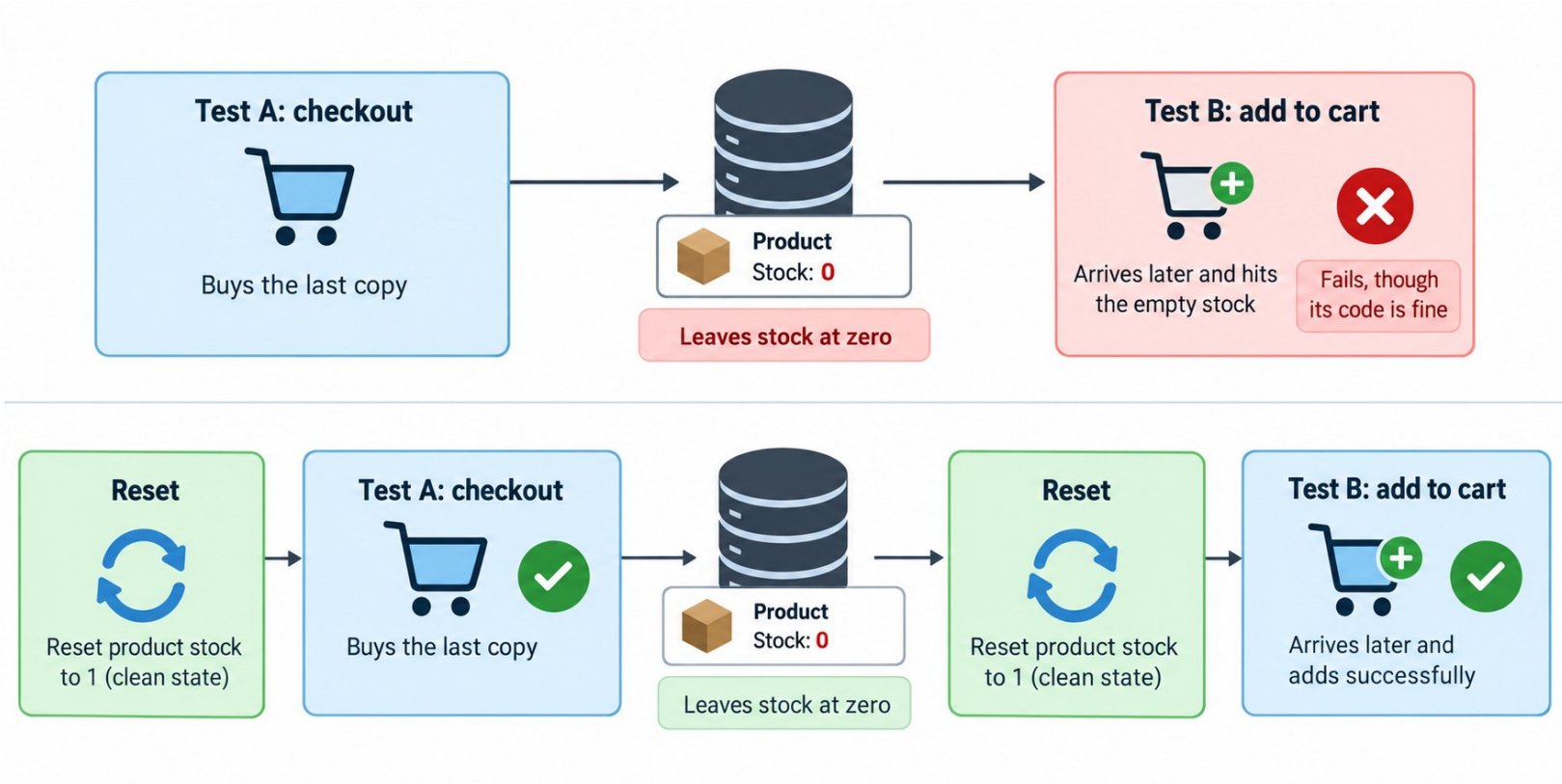
SHARED STATE

Some state lives **outside** the test: on the server, in a database, or in an account. One test changes it, and the next test inherits the change.

A checkout test buys the last copy of a book. A later test that adds the same book to the cart fails, though its code is fine. Reset shared state **before each test**, or give each test its own data.

© 2026 by Innovation In Software Corporation

SHARED STATE



© 2026 by Innovation In Software Corporation

RANDOM AND UNCONTROLLED DATA

"Random data gives more coverage" sounds smart. In practice, a random test checks a different thing each run, can't be reproduced, and fails only when the dice land on a special case.

- Choose specific data that represents each case you care about
- Test the special cases (out of stock, empty, the maximum) on purpose, in their own tests
- If you need many values, use a data-driven test over a fixed list

© 2026 by Innovation In Software Corporation

TIME, NETWORK, AND ENVIRONMENT

Anything a test doesn't control can make it flaky.

Source	Example	Fix
Clock	A code that expires "today"	Pass the date in; control it in the test
Network speed	A slow response on a busy runner	Wait for the request, not for time
External services	A third-party API is down	Mock it in most tests
Machine differences	Fonts render differently on Linux	Make baselines in the environment that runs the tests

© 2026 by Innovation In Software Corporation

DETECTING FLAKY TESTS

Flakiness hides on a fast laptop. Go looking for it.

Technique	How	Finds
Rerun	Run the same suite several times	Anything intermittent
Isolate	Run one test on its own (`it.only`)	Order dependence and shared state
Burn-in	Repeat one test many times in a single run	Low-probability flakes, like random data
Slow it down	Add artificial delay to the app	Timing problems hidden by a fast machine
Track history	Record which tests fail on unchanged code	Repeat offenders across many runs

© 2026 by Innovation In Software Corporation

BURN-IN TESTING

One green run proves nothing about a flaky test. Run it many times in a row.

```
Cypress._.times(20, run => {
  it(`adds a book to the cart (run ${run + 1})`, () => {
    // ...the test body, unchanged
  });
});
```

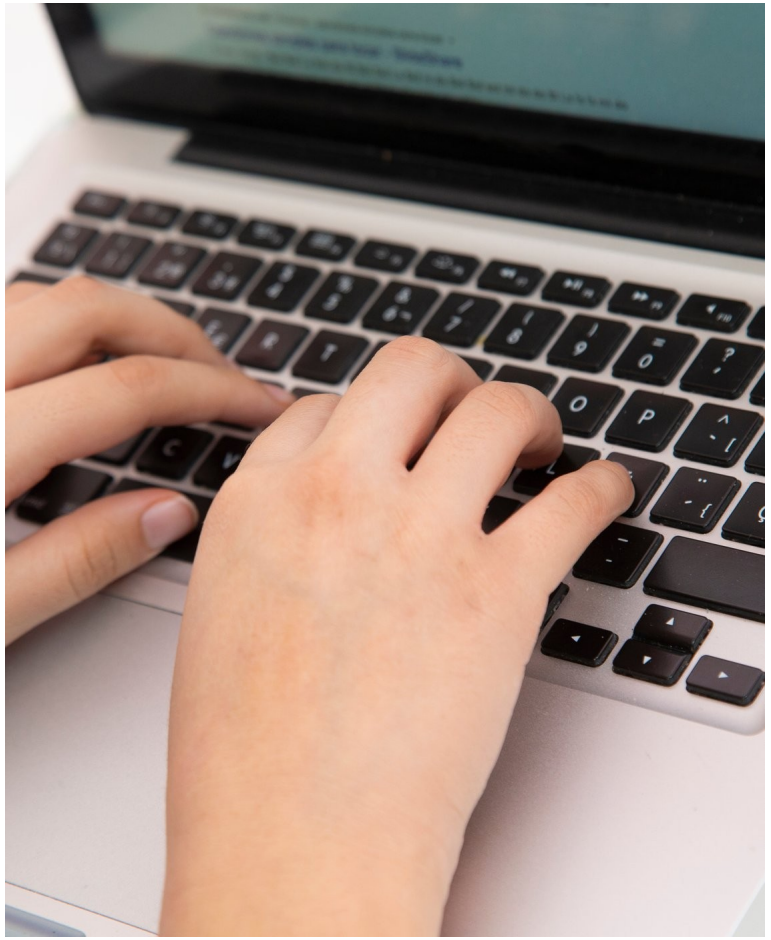
If any of the 20 fail, the test is flaky. After a fix, burn it in again to prove it's reliable, ideally under the same conditions that made it fail.

© 2026 by Innovation In Software Corporation

FAULT INJECTION: MAKE IT WORSE ON PURPOSE

Fault injection deliberately makes the environment worse to see what breaks: slow responses, failed requests, a dependency that's down.

It exposes fragile **tests** (a fixed wait that's suddenly too short) and fragile **apps** (a page that never shows an error when a request fails). Adding a random delay to every request is the simplest version.



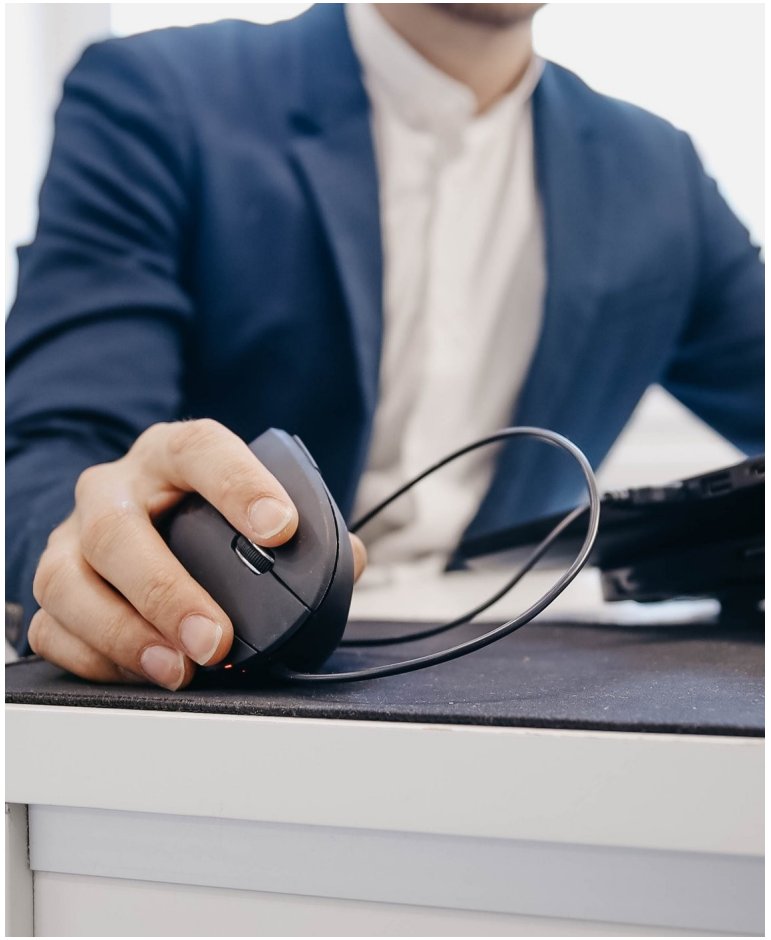
© 2026 by Innovation In Software Corporation

WHY RETRIES AREN'T A FIX

Most frameworks can retry a failed test automatically. It turns the build green, so it's tempting.

- The test is still flaky; you've just hidden it
- Real **intermittent bugs** in the app get retried away too
- Runs get slower as retries pile up
- Retries are a short-term bandage while you fix the cause, never the fix itself

The same goes for raising every timeout. A longer guess is still a guess.



© 2026 by Innovation In Software Corporation

TEST RELIABILITY VS. APP RELIABILITY

"Reliable" means two different things in QA.

	Test reliability	App reliability
Question	Does the test give the same answer for the same code?	Does the app keep working over time and when things go wrong?
Examples	Fixed waits, shared state, random data	Uptime, recovery after errors, long-running stability
How you test it	Reruns, isolation, burn-in	Soak tests, fault injection, monitoring

A flaky test and a fragile app often show up together when the environment gets slow.

© 2026 by Innovation In Software Corporation

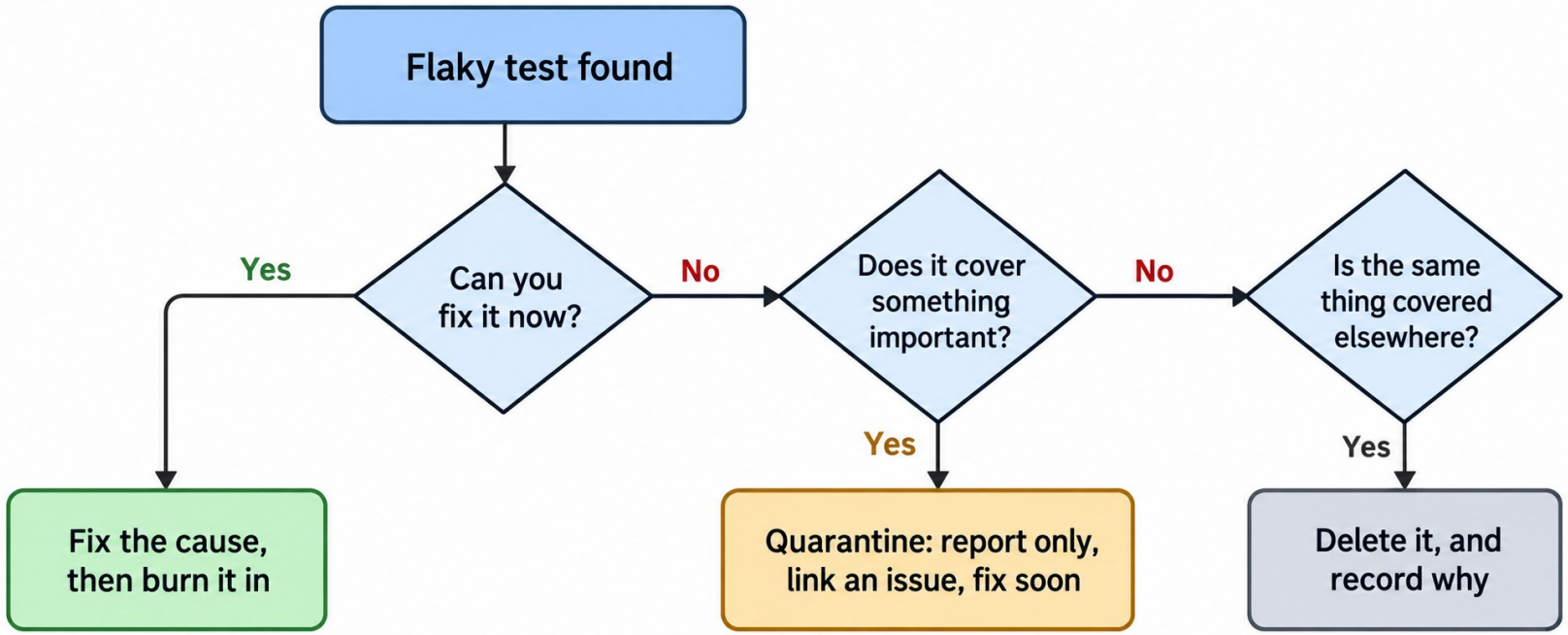
TIMING ASSERTIONS: FLAKY OR SLOW?

A check like "responds in under 500 ms" is useful as an early warning, but it's only as stable as the machine it runs on. When it fails, ask whether the **test** is flaky or the **app** got slower.

- Look at many samples, not one: averages and percentiles over time
- Check the environment: was the runner busy? Did unrelated tests slow down too?
- Check the server: logs, response times per endpoint, recent changes
- Measure real performance in a controlled environment with dedicated tools

© 2026 by Innovation In Software Corporation

FIX, QUARANTINE, OR DELETE?



© 2026 by Innovation In Software Corporation

OVER-RELIANCE ON MANUAL TESTING

Some teams run the same regression checklist by hand before every release. It takes days, people skip steps when they're tired, and it gets longer every sprint.

- Manual regression doesn't scale
- Human attention is best spent on exploration and judgment
- Automate the stable, repetitive, high-risk checks
- Keep people for what machines can't do: exploring, questioning, and evaluating the experience



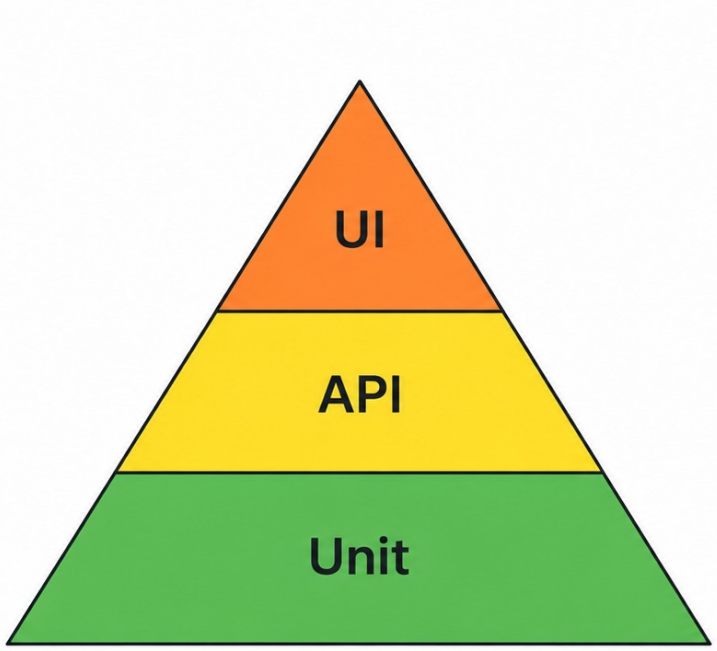
© 2026 by Innovation In Software Corporation

THE ICE-CREAM CONE

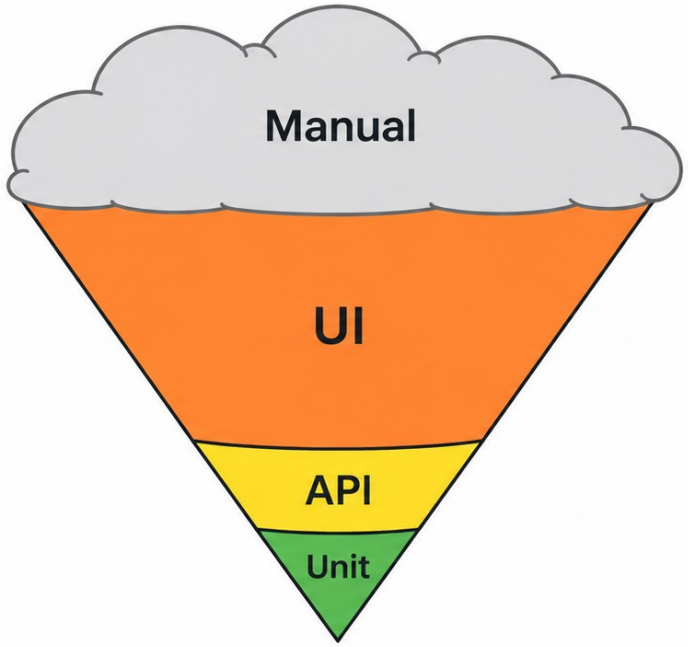
The opposite mistake is automating everything through the UI. The test pyramid turns upside down into an **ice-cream cone**: a few unit tests, some API tests, a huge layer of slow UI tests, and manual testing on top.

© 2026 by Innovation In Software Corporation

THE ICE-CREAM CONE



Fast, stable, cheap



Slow, flaky, expensive

© 2026 by Innovation In Software Corporation

AUTOMATING THE WRONG THINGS

Automation isn't automatically good. Automating the wrong tests costs more than it saves.

- Features that change every week
- Checks only a person can judge, like whether a page "looks right" or wording is clear
- One-off tests that will run once
- Everything at the UI level, when an API or unit test could prove it faster

Automate what runs often, matters most, and is stable, at the lowest level that can prove it.

© 2026 by Innovation In Software Corporation

IGNORING NON-FUNCTIONAL REQUIREMENTS

Functional bugs are easy to see. Slow pages, security gaps, inaccessible forms, and unreliable systems often go untested until a customer, an auditor, or a lawsuit finds them.

- Vague requirements ("should be fast") get skipped because nobody can test them
- Push for measurable targets: response time percentiles, WCAG level, uptime
- Plan non-functional testing from the first test plan, not after launch



© 2026 by Innovation In Software Corporation

MISCOMMUNICATION BETWEEN QA AND DEVELOPMENT

"QA just throws bugs over the wall."
"Developers never tell us what changed."
When QA and development talk past each other, bugs bounce, fixes miss, and blame replaces collaboration.

The cause is usually the process, not the people: work handed off instead of shared, and nobody in the room when decisions are made.



© 2026 by Innovation In Software Corporation

BRIDGING THE GAP

Practice	Helps because
Three Amigos before a story starts	Shared understanding of "done"
Clear **pull request** descriptions	QA knows what changed and what to test
Factual bug reports with evidence	Fewer questions, faster fixes
Blameless language	People stay open instead of defensive
Pairing on hard bugs	Two perspectives, one fix
Shared **quality goals**	Quality is the team's job, not QA's alone

© 2026 by Innovation In Software Corporation

STRATEGIES TO AVOID COMMON QA MISTAKES

Mistake	Strategy	Where you practiced it
Testing without priorities	Risk-based test plan	Lab 1.1
Only scripted, or only exploratory	Use both	Lab 1.2
Untestable requirements	Acceptance criteria, Three Amigos	Lab 1.3
Bugs found late	TDD, shift left, CI	Labs 2.1, 3.3
Upside-down test pyramid	Push checks down to unit and API	Labs 2.2, 2.4
Ignored non-functional needs	Accessibility, security, and performance checks	Labs 2.2–2.4
Vague bug reports	Templates and evidence	Lab 3.2
Flaky, untrusted pipelines	Reliable patterns, quarantine, fast fixes	Today

© 2026 by Innovation In Software Corporation

BUILDING A QUALITY CULTURE

Tools and techniques only work in a team that values quality. A quality culture is one where everyone owns quality, problems are raised early without blame, and a red build stops the line.

QA's role in that culture isn't to be the gatekeeper at the end. It's to be the person who helps the whole team build quality in from the start.



© 2026 by Innovation In Software Corporation

LAB 3.4 – FIX THE FLAKY SUITE

A developer wrote a quick shop test suite before beta. It "passes on my machine," but fails in CI about one run in three. Your job: work out why, and fix it.

- Rerun the suite, slow BookNook down, isolate single tests, and burn in a suspect
- Diagnose five tests, each with a different antipattern
- Fix them without fixed waits, longer timeouts, or retries
- Prove the fix with a burn-in on a slow server, then audit the team for wider QA antipatterns

© 2026 by Innovation In Software Corporation

POP QUIZ:

A test clicks a button, waits 500 ms, then checks the result once. It fails on busy CI machines. What's the best fix?

A. Increase the wait to 3000 ms

B. Turn on automatic retries

C. Replace the wait with an assertion that retries until the condition is true

D. Run the test only on fast machines



POP QUIZ:

A test clicks a button, waits 500 ms, then checks the result once. It fails on busy CI machines. What's the best fix?

A. Increase the wait to 3000 ms

B. Turn on automatic retries

C. Replace the wait with an assertion that retries until the condition is true

D. Run the test only on fast machines





POP QUIZ:

A test passes every time you run it alone, but fails when the whole suite runs. What's the most likely cause?

A. The test framework is broken

B. Shared state or order dependence between tests

C. The browser is out of date

D. The test is too short





POP QUIZ:

A test passes every time you run it alone, but fails when the whole suite runs. What's the most likely cause?

A. The test framework is broken

B. Shared state or order dependence between tests

C. The browser is out of date

D. The test is too short



POP QUIZ:

Why aren't automatic retries a real fix for flaky tests?

A. They make the pipeline too fast

B. They hide the flakiness, and real intermittent bugs get retried away too

C. Cypress doesn't support retries

D. Retries only work for unit tests



POP QUIZ:

Why aren't automatic retries a real fix for flaky tests?

A. They make the pipeline too fast

B. They hide the flakiness, and real intermittent bugs get retried away too

C. Cypress doesn't support retries

D. Retries only work for unit tests



DAY 3 WRAP-UP

Today you took tests from "they found something" to "the team can trust them."

- Triage turns symptoms into reproducible, isolated defects with evidence
- Actionable bug reports and honest metrics drive good release decisions
- CI runs every test on every change; QA designs what blocks a merge
- Flaky tests and team antipatterns erode trust, and both can be fixed

© 2026 by Innovation In Software Corporation

COURSE WRAP-UP

In three days you followed one thread from requirements to a trusted pipeline.

- **Day 1:** plan by risk, explore and script, and write test cases anyone can run
- **Day 2:** turn those test cases into unit, UI, and API automation
- **Day 3:** triage what tests find, report it clearly, and run it all in CI

Quality isn't a phase or a department. It's a habit the whole team practices, and QA is the person who keeps it going.



© 2026 by Innovation In Software Corporation